NE 723: Computer Project 1

Joseph F. Specht IV

October 13, 2026

1 Algorithm Flowchart

For this section, I decided to report the steps with every relevant decision I had to make. I will also include the function name used to perform the action. For example, to generate the quadrature set, I call the function "discretize_angle()" which is in the file "quadrature.py," so I will report "quadrature.discretize_angle()."

I will also note, I am solving the following linear system to obtain the angular flux $\psi_{m,i}$:

$$A\psi_{m,i} = b \tag{1}$$

$$A = \mu_m(L_{m,i} + C) + \Sigma_{t,i}\Delta x_i M_i \tag{2}$$

$$b = \frac{1}{2}\left(\Sigma_{s,i} + \nu_{f,i}\Sigma_{f,i}\right)\Delta x_i M_i \phi_i + \frac{\Delta x_i}{2} M_i q_i + D \tag{3}$$

where, for $\mu > 0$,

$$C = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix} \tag{4}$$

$$D = \begin{pmatrix} \psi_{L,m,i+1} & 0 \\ 0 & 0 \end{pmatrix} \tag{5}$$

for $\mu < 0$,

$$C = \begin{pmatrix} -1 & 0 \\ 0 & 0 \end{pmatrix} \tag{6}$$

$$D = \begin{pmatrix} 0 & 0 \\ 0 & \psi_{R,m,i-1} \end{pmatrix} \tag{7}$$

1. Problem Setup

   (a) Read input file. My input files are python files, so the inputs can easily be imported like: "import test_{a,b,c} as problem".

   (b) Discretize space and express cross sections, ν_f, and q in the same size array as space. mesh_builder.discretize_space()

   (c) Discretize angle by generating the quadrature set. quadrature.discretize_angle()

2. Determine Problem Type. If the problem is a fixed source problem, continue to step 3A. If the problem is a criticality search problem, continue to step 3B.

3A. Solve Fixed Source Problem
solvers.fixed_source_solver()

   (a) Perform the angular mesh sweep
   solvers.angular_mesh_sweep()

      i. Compute b (Eq. 3) for the current iteration. The vector b is the sum of (i) the internal source (sum of scattering and fission) as the sum of the internal source (scattering and fission), (ii) the external source, and (iii) the known upwinding condition.

      ii. Compute A (Eq. 2) for the current iteration. The matrix A is the sum of (i) the streaming term, (ii) the unknown upwinding condition, and (iii) the total interaction term.

      iii. Compute the determinant of A.

      iv. Compute $\psi_{L,m,i}$ as:

$$\psi_{L,m,i} = \frac{1}{\det A}(b_1 * a_{22} - b_2 * a_{12}) \tag{8}$$

      v. Compute $\psi_{R,m,i}$ as:

$$\psi_{R,m,i} = \frac{1}{\det A}(b_2 * a_{11} - b_1 * a_{21}) \tag{9}$$

      vi. Compute the cell averaged angular flux:

$$\bar{\psi}_{m,i} = \frac{1}{2}(\psi_{L,m,i} + \psi_{R,m,i}) \tag{10}$$

      vii. Compute the first moment of the angular flux:

$$\hat{\psi}_{m,i} = \frac{1}{2}(\psi_{R,m,i} - \psi_{L,m,i}) \tag{11}$$

viii. Compute the cell-edge angular flux:

$$\psi_{m,i+1/2} = \psi_{R,m,i} \quad \text{for } \mu_m > 0 \tag{12}$$

$$\psi_{m,i-1/2} = \psi_{L,m,i} \quad \text{for } \mu_m < 0 \tag{13}$$

ix. Add the contributions of the current ordinate to the scalar flux and the current:

$$\phi_{L/R,i} = \phi_{L/R,i} + w_m \psi_{L/R,m,i} \tag{14}$$

$$J_{L/R,i} = J_{L/R,i} + w_m \mu_m \psi_{L/R,m,i} \tag{15}$$

(b) Compute $||\phi^{(l)} - \phi^{(l-1)}||_\infty$ and the estimation of the spectral radius of iterations.

$$\rho_\phi^{(l)} = \frac{||\phi^{(l)} - \phi^{(l-1)}||_\infty}{||\phi^{(l-1)} - \phi^{(l-2)}||_\infty} \tag{16}$$

(c) Evaluate convergence criteria, which is given below. If the convergence criteria is not met, return to step 3A.a. If the convergence criteria is met, continue to step 3A.d.

$$||\phi^{(l)} - \phi^{(l-1)}||_\infty < \epsilon_\phi \left(\frac{1}{\rho_\phi^{(l)}} - 1 \right) \tag{17}$$

(d) Perform post-processing to obtain required quantities.

i. Compute the cell-average scalar flux and cell-average current:

$$\bar{\phi}_i = \frac{1}{2}(\phi_{R,i} + \phi_{L,i}) \tag{18}$$

$$\bar{J}_i = \frac{1}{2}(J_{R,i} + J_{L,i}) \tag{19}$$

ii. Compute the first Legendre spatial moment of the scalar flux and current:

$$\hat{\phi}_i = \frac{1}{2}(\phi_{R,i} - \phi_{L,i}) \tag{20}$$

$$\hat{J}_i = \frac{1}{2}(J_{R,i} - J_{L,i}) \tag{21}$$

iii. Sum the upwind, grid-edge angular fluxes to find the grid-edge scalar flux and current:

$$\phi_{i+1/2} = \sum_{m=1}^{\text{l ord}} w_m \psi_{m,i+1/2} \tag{22}$$

$$J_{i+1/2} = \sum_{m=1}^{\text{l ord}} w_m \mu_m \psi_{m,i+1/2} \tag{23}$$

iv. Compute the scheme and iteration residuals.

3B. Solve Criticality Search Problem

solvers.critical_width_solver()

(a) Guess initial slab with, a_0 and calculate the multiplication factor for a_0 and $a_1 = 1.15a_0$ (the constant is relatively arbitrary, but it should not be too much bigger than a_0.

solvers.eigenvalue_inner_loop()

(b) Begin outer criticality loop using.

i. Calculate the next width guess using the secant method for root finding.

$$a^{(j+1)} = a^{(j)} - \frac{(k^{(j)} - 1)(a^{(j)} - a^{(j-1)})}{k^{(j)} - k^{(j-1)}} \tag{24}$$

ii. Regenerate the spatial mesh

mesh_builder.discretize_space()

iii. Execute the inner eigenvalue loop using power iteration

A. Start with an initial flux guess of $\vec{1}$ (a vector of ones) and a k guess of 1.

B. Scale the production coefficient to use the current guess for k.

$$\Sigma_s + \frac{\nu_f \Sigma_f}{k^{(l-1)}} \tag{25}$$

C. Call "angular_mesh_sweep()" (Step 3A.a)

D. Make a new guess for the k-eigenvalue.

$$k^{(l)} = k^{(l)} \frac{\sum \nu_f \bar{\phi}^{(l)} \Delta x}{\sum \nu_f \bar{\phi}^{(l-1)} \Delta x} \tag{26}$$

E. Evaluate inner convergence criteria.

$$||\phi^{(l)} - \phi^{(l-1)}||_\infty < \epsilon_\phi \left(\frac{1}{\rho_\phi^{(l)}} - 1 \right) \tag{27}$$

$$||k^{(l)} - k^{(l-1)}||_\infty < \epsilon_k \left(\frac{1}{\rho_k^{(l)}} - 1 \right) \tag{28}$$

$$\rho_\phi^{(l)} = \frac{||\phi^{(l)} - \phi^{(l-1)}||_\infty}{||\phi^{(l-1)} - \phi^{(l-2)}||_\infty} \tag{29}$$

F. If the convergence criteria is not met, return to Step 3B.b.iii.A, but using the current flux and k-eigenvalue as the next flux guess and the k-eigenvalue guess instead of the initial guesses. If the convergence criteria is met, continue to Step 3B.b.iii.G.

G. Normalize the flux such that the integral is equal to 1.

$$\phi = \frac{\phi}{\sum \bar{\phi} \Delta x} \tag{30}$$

H. Perform post-processing to obtain the same quantities as calculated in Step 3A.d.

4. Write the results to output files.

5. Generate plots using the output files.

2 Test A

For this test, the number of source iterations to converge was 44.

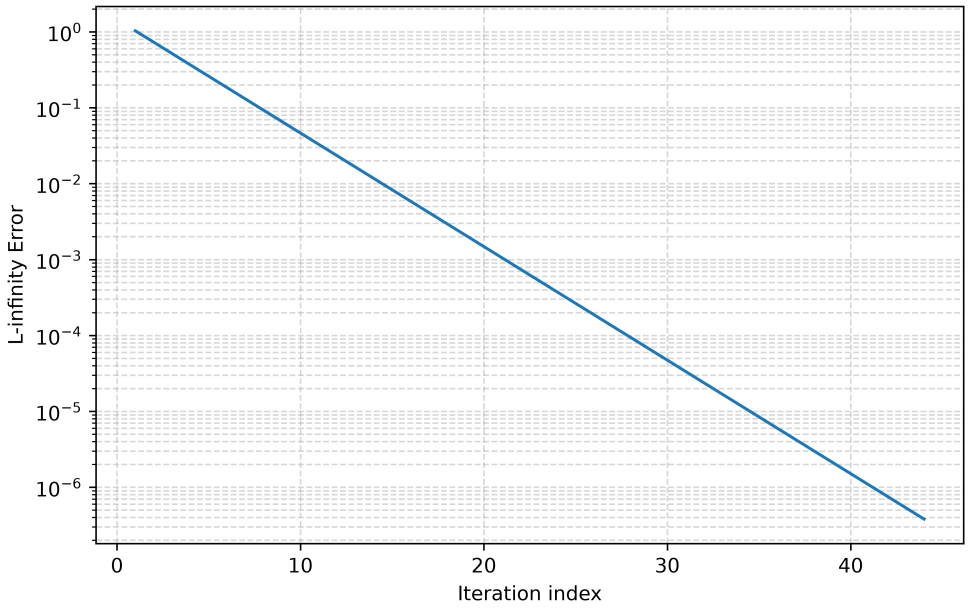


Figure 1: L infinity norm vs iteration index.

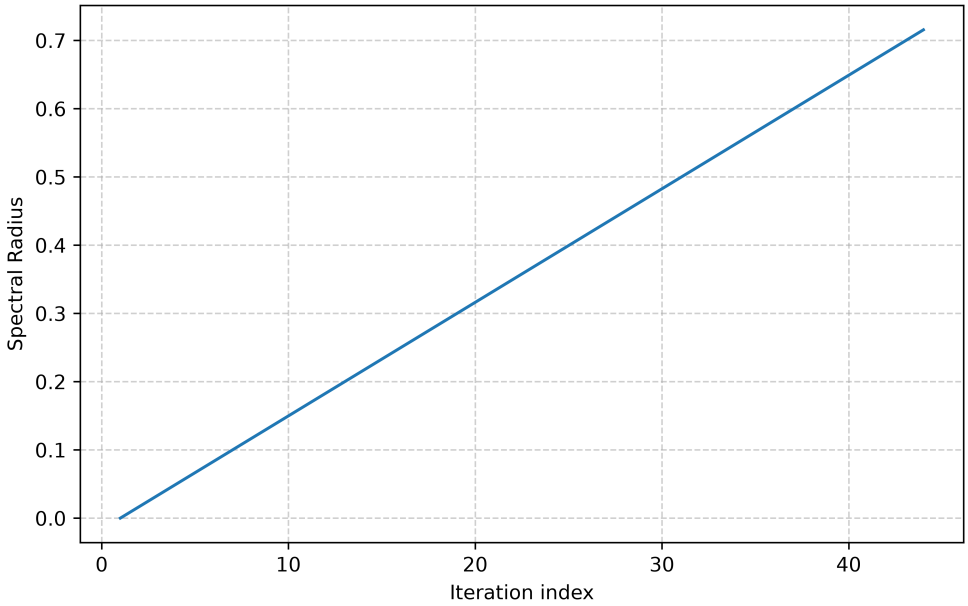


Figure 2: Spectral radius vs iteration index.

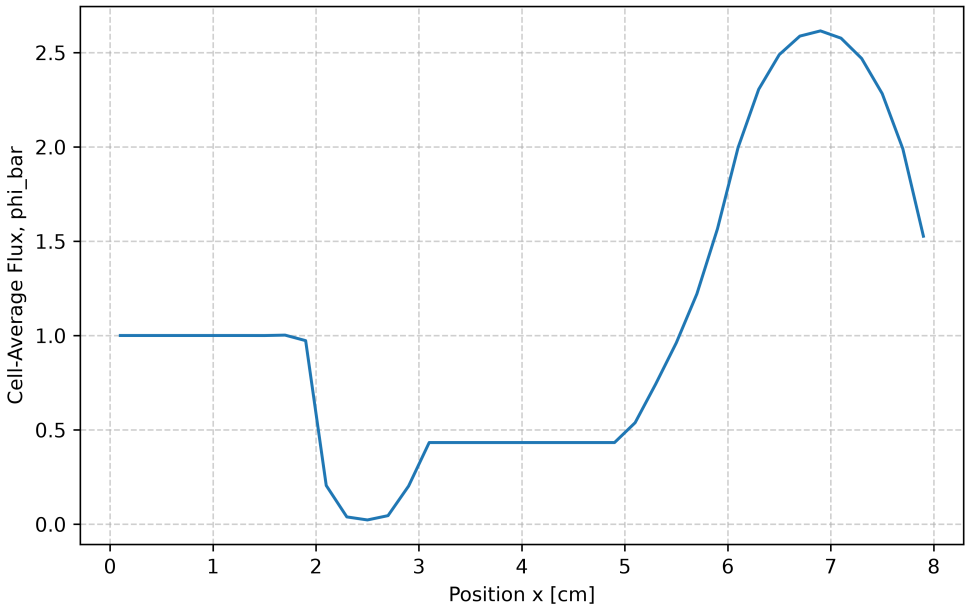


Figure 3: Cell-average scalar flux vs position.

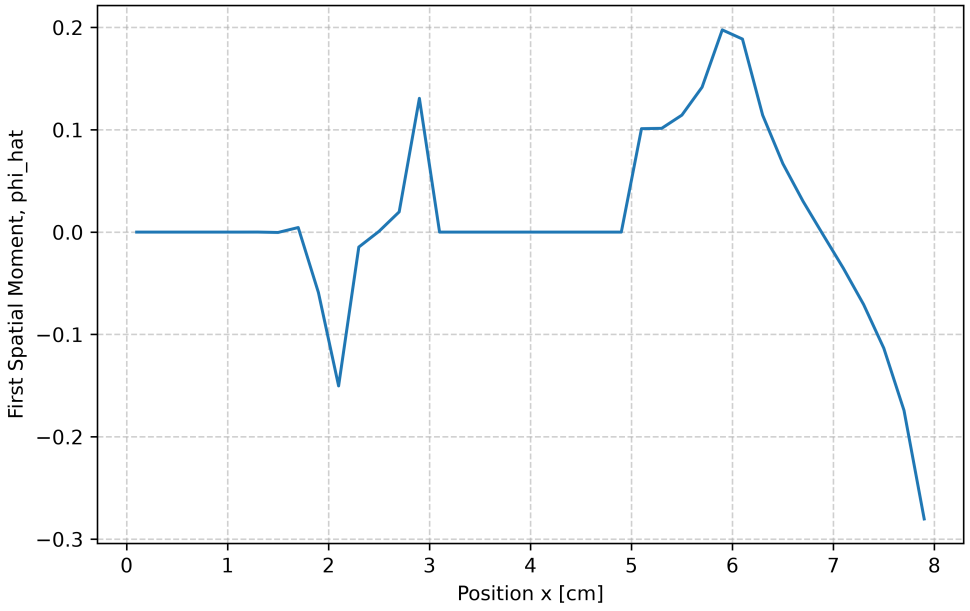


Figure 4: First spatial moment of the scalar flux vs position.

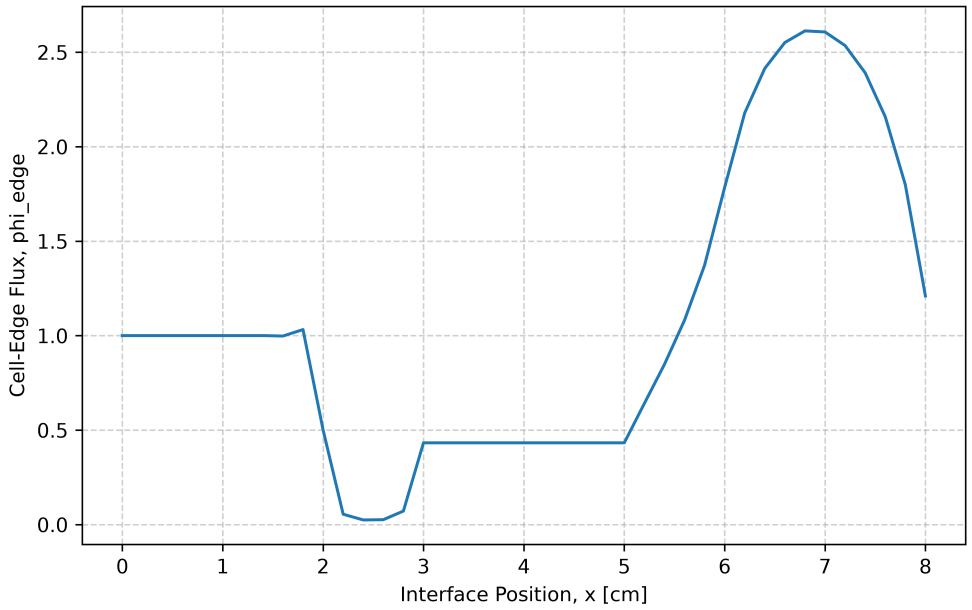


Figure 5: Cell-edge values of the scalar flux vs position.

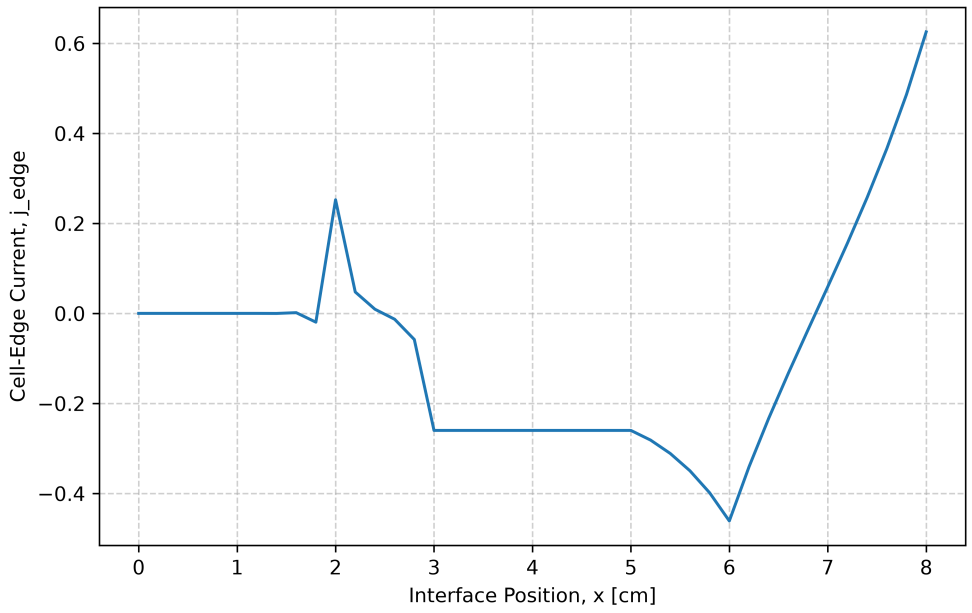


Figure 6: Cell-edge values of the current vs position.

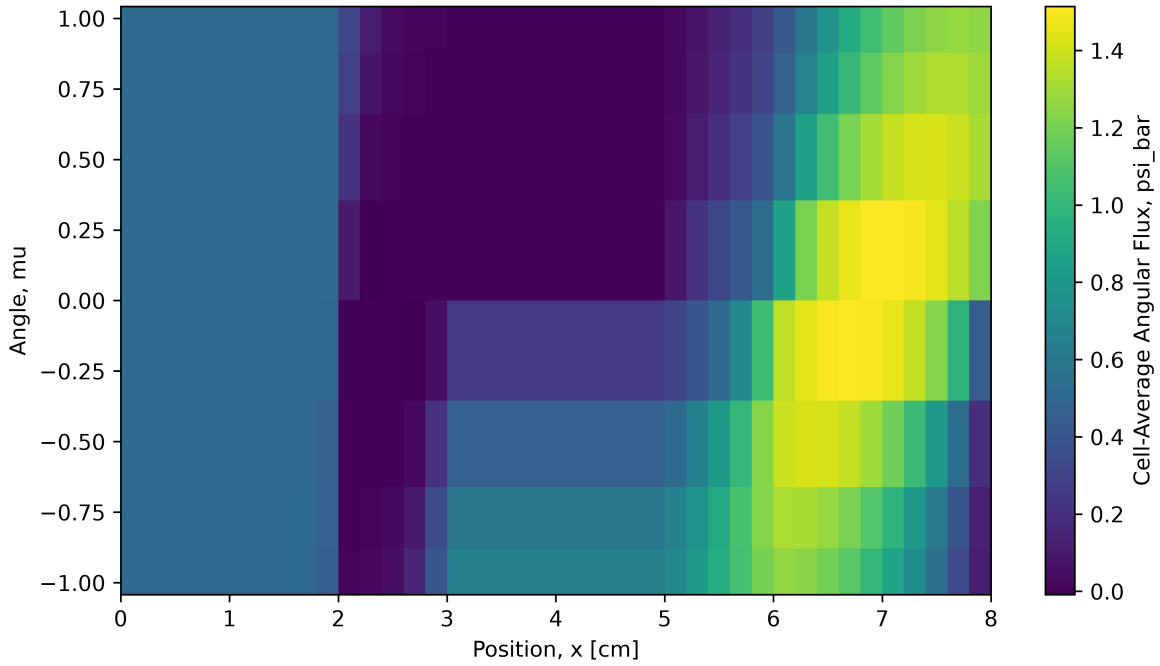


Figure 7: Cell-average values of the angular flux vs position and angle.

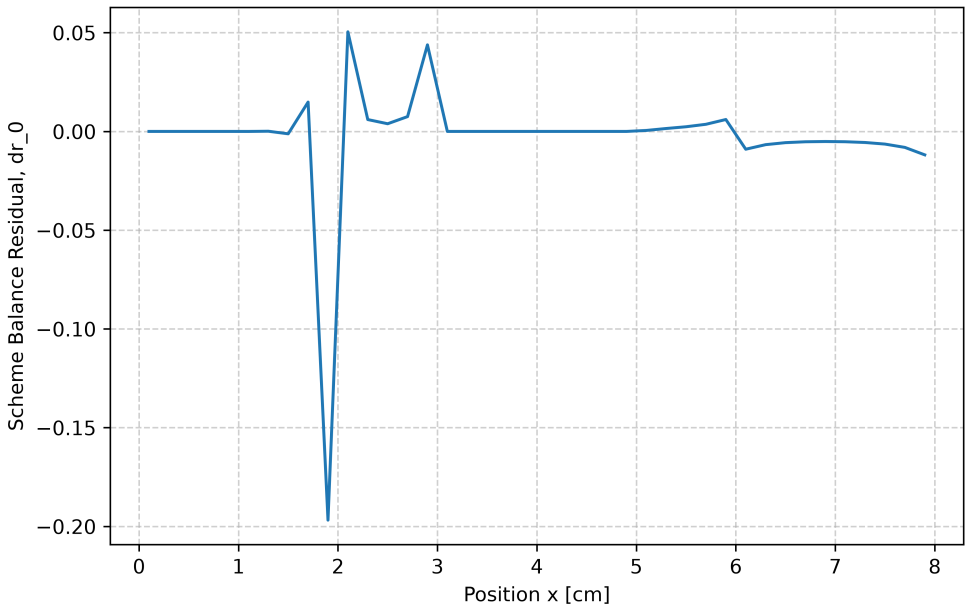


Figure 8: Scheme residual for the particle balance equation vs position.

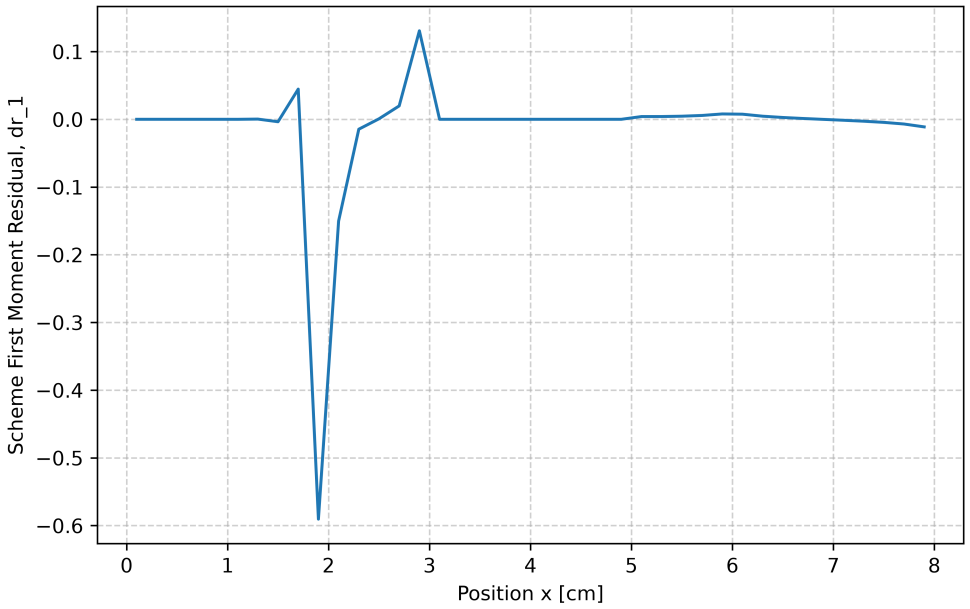


Figure 9: Scheme residual for the first moment of the particle balance equation vs position.

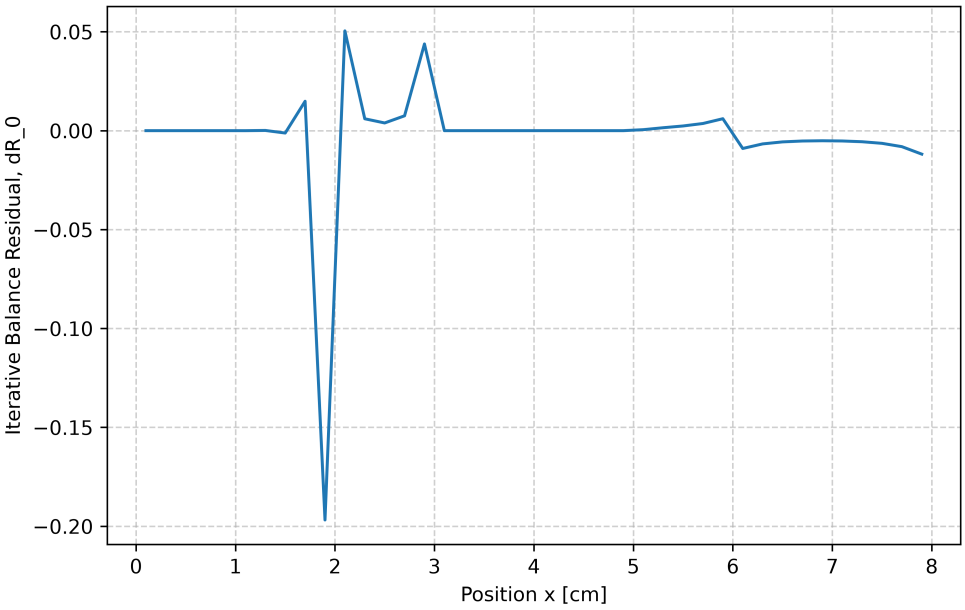


Figure 10: Iterative residual for the particle balance equation vs position.

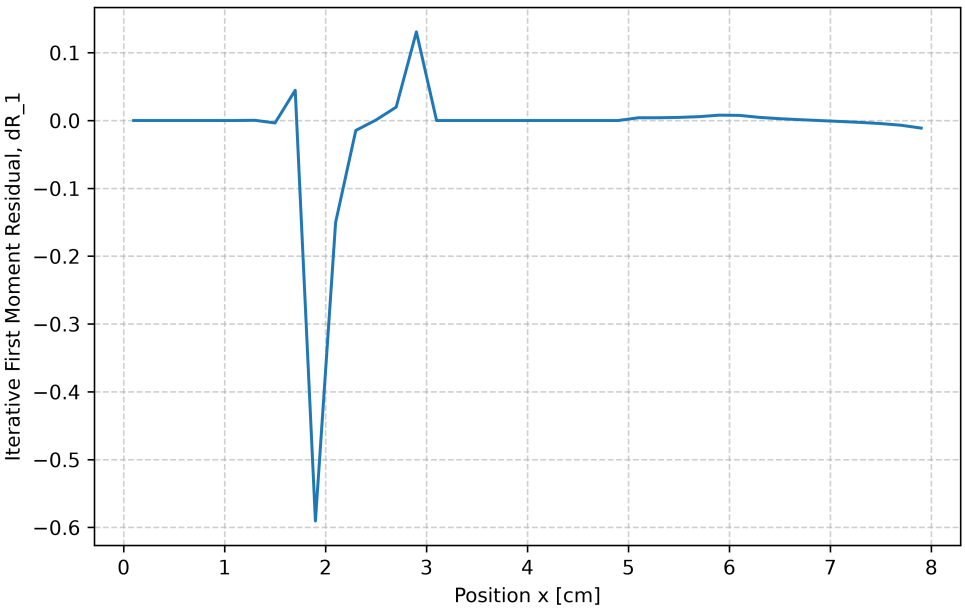


Figure 11: Iterative residual for the first moment of the particle balance equation vs position.

3 Test B

For this test, the number of source iterations to converge was 3,134,275.

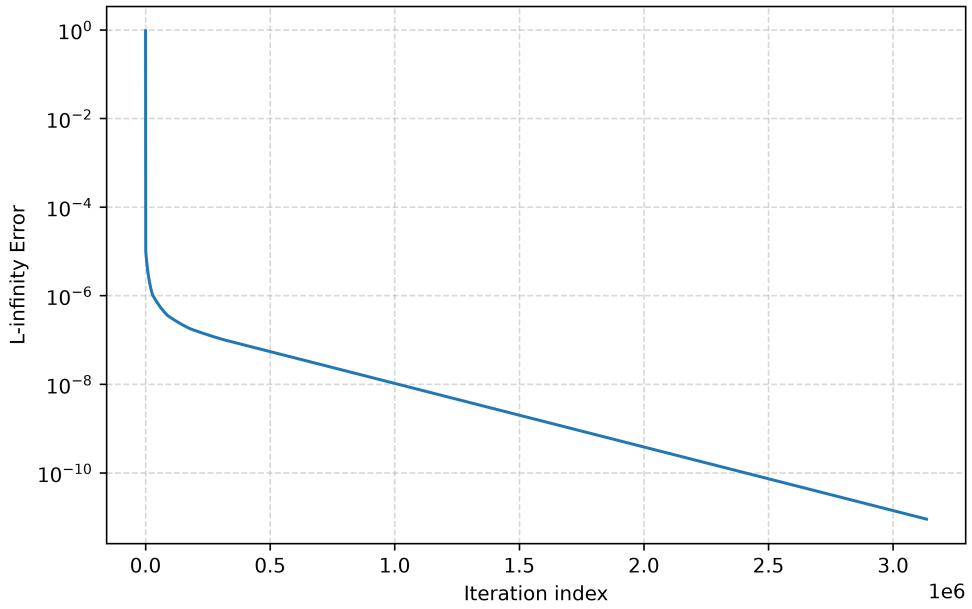


Figure 12: L infinity norm vs iteration index.

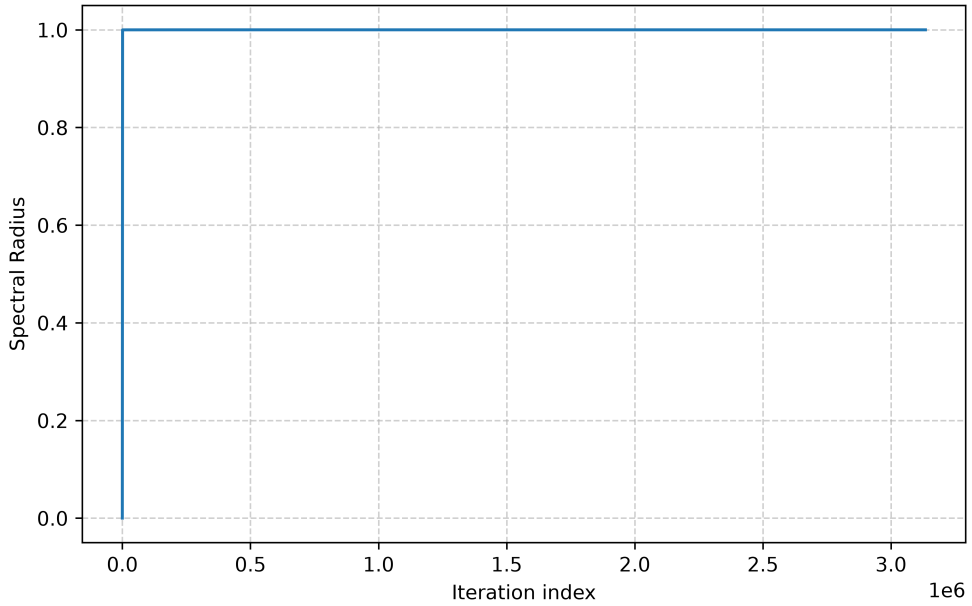


Figure 13: Spectral radius vs iteration index.

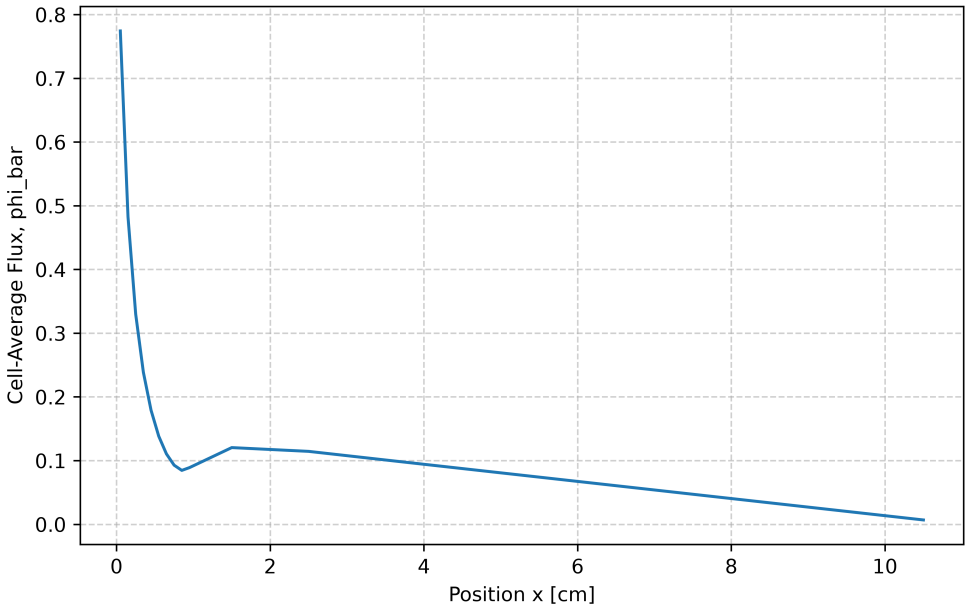


Figure 14: Cell-average scalar flux vs position.

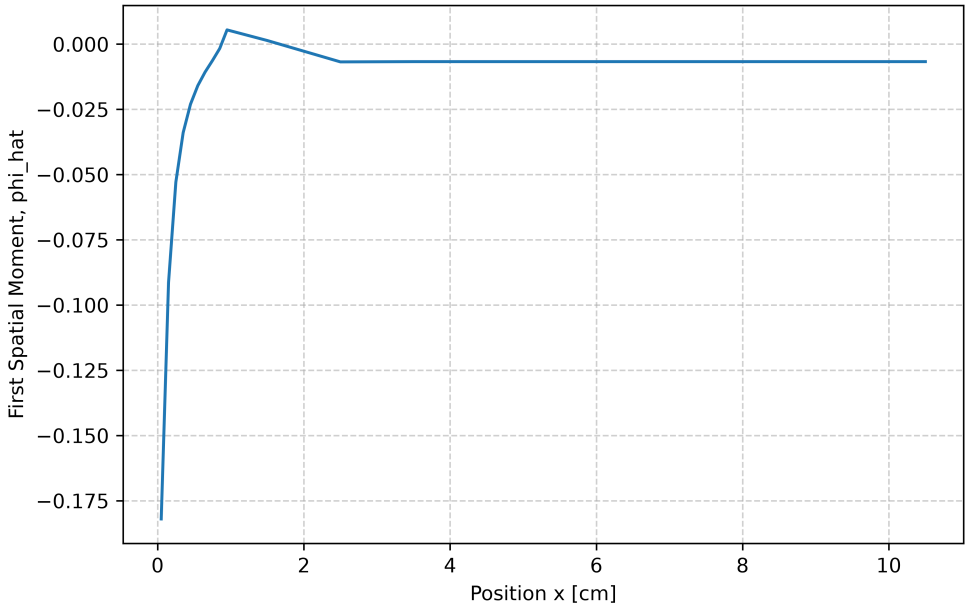


Figure 15: First spatial moment of the scalar flux vs position.

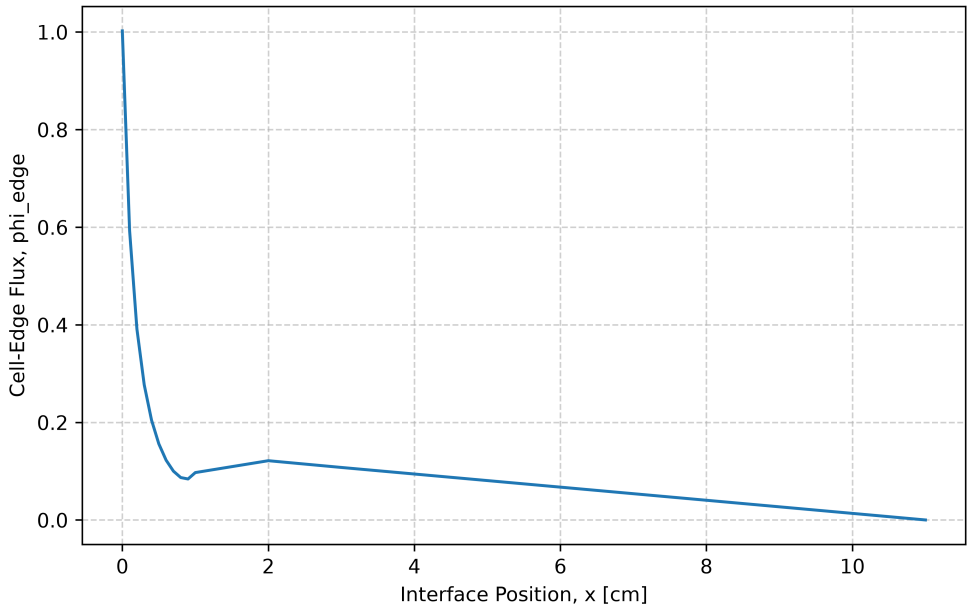


Figure 16: Cell-edge values of the scalar flux vs position.

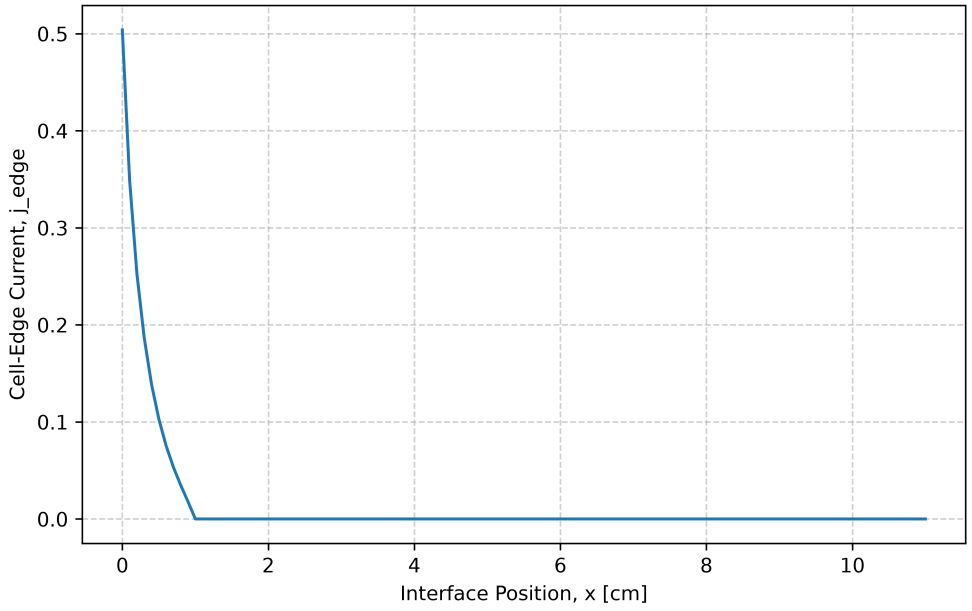


Figure 17: Cell-edge values of the current vs position.

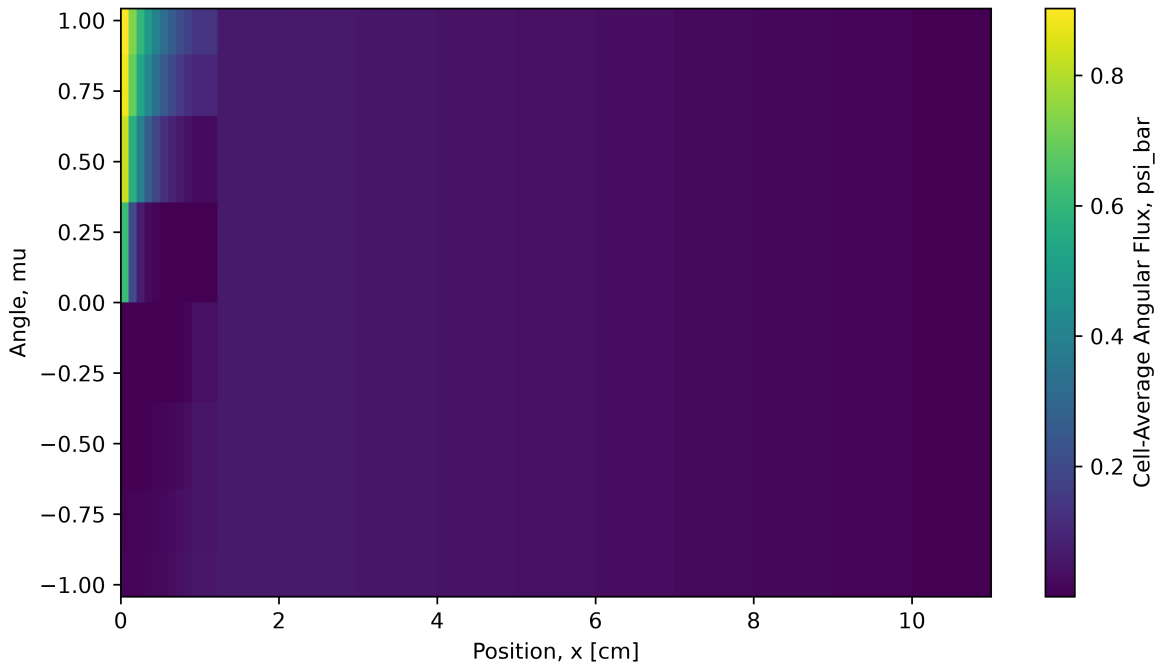


Figure 18: Cell-average values of the angular flux vs position and angle.

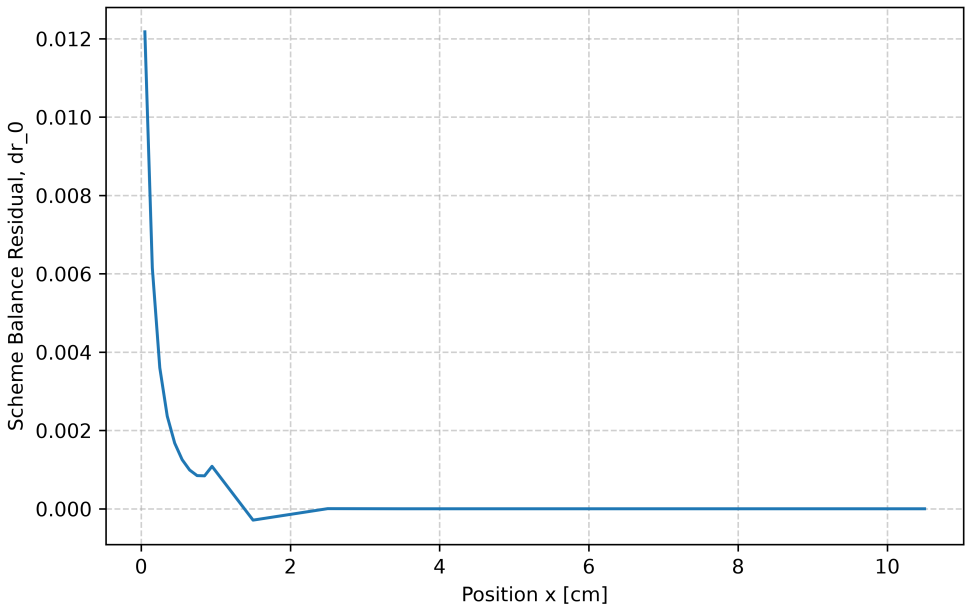


Figure 19: Scheme residual for the particle balance equation vs position.

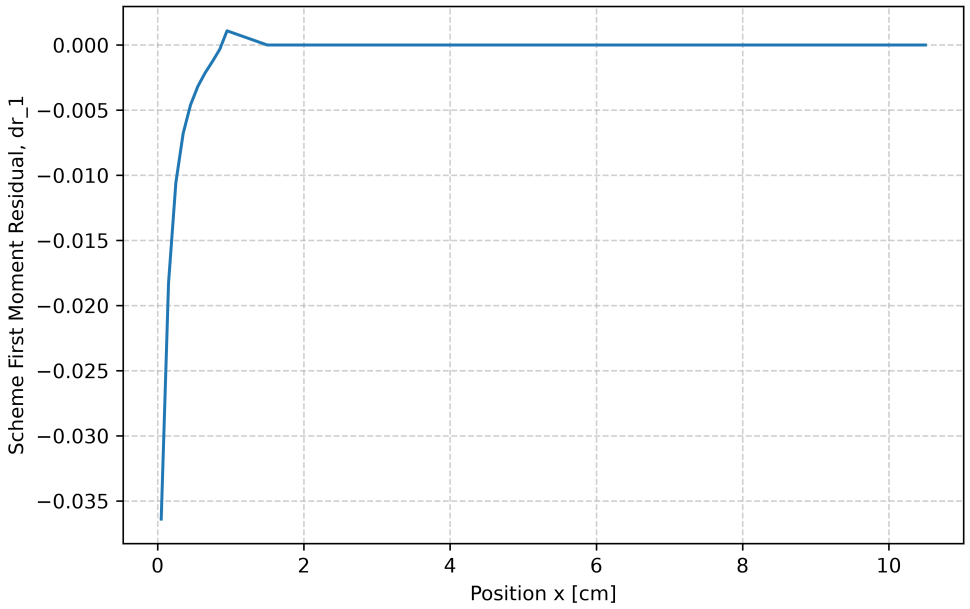


Figure 20: Scheme residual for the first moment of the particle balance equation vs position.

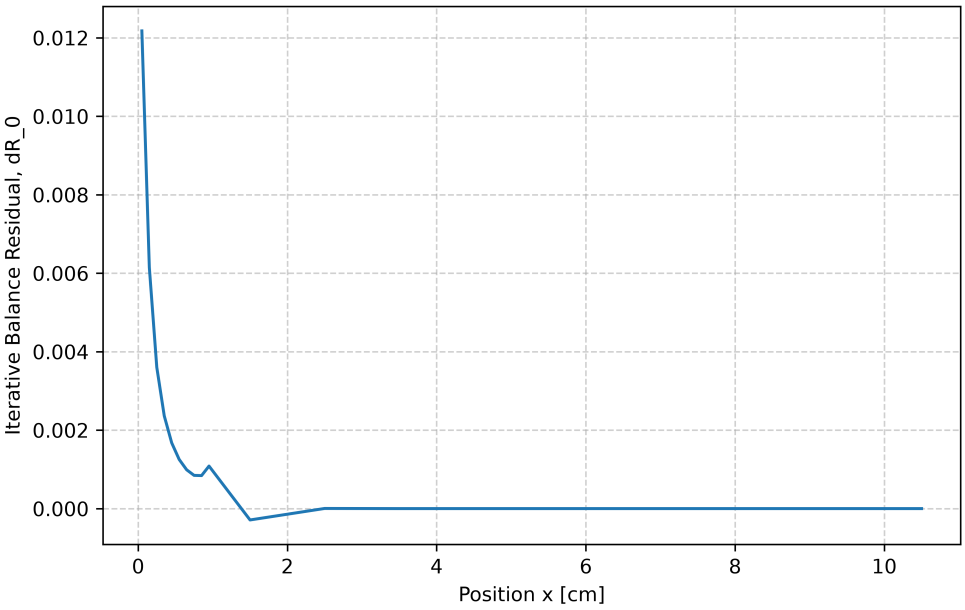


Figure 21: Iterative residual for the particle balance equation vs position.

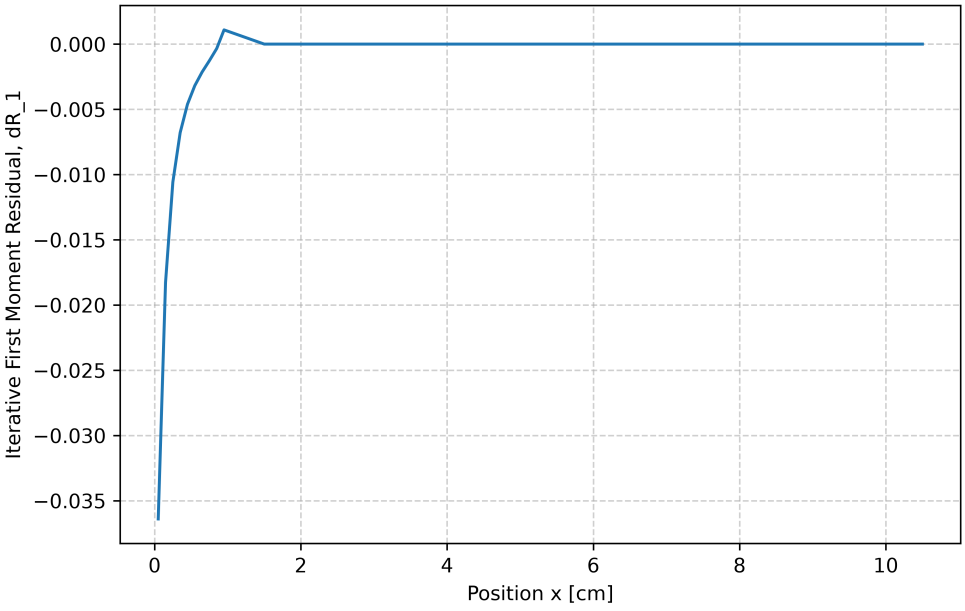


Figure 22: Iterative residual for the first moment of the particle balance equation vs position.

4 Test C

For this test, the final critical slab width was 6.60777 cm.

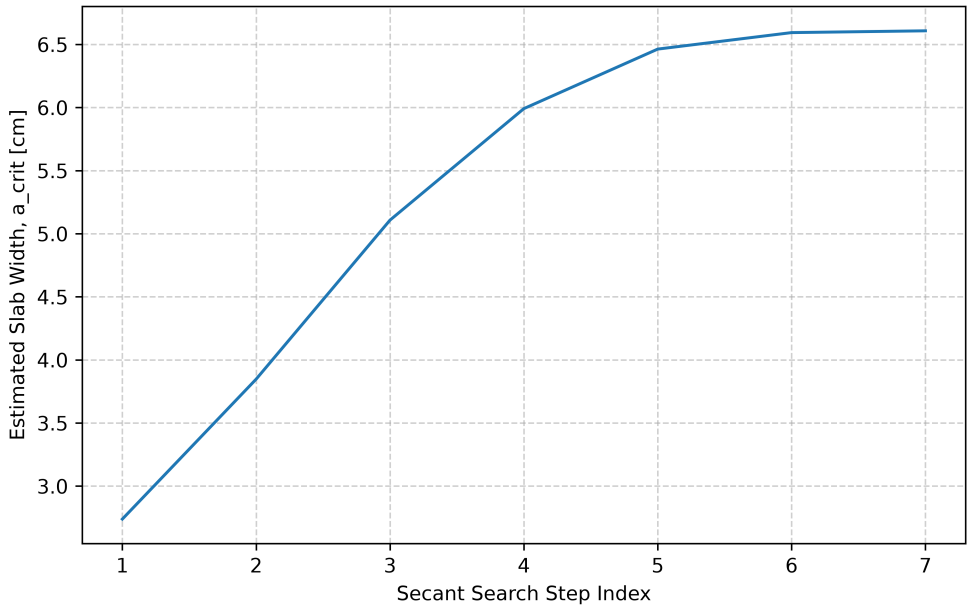


Figure 23: Critical with history vs outer iteration index.

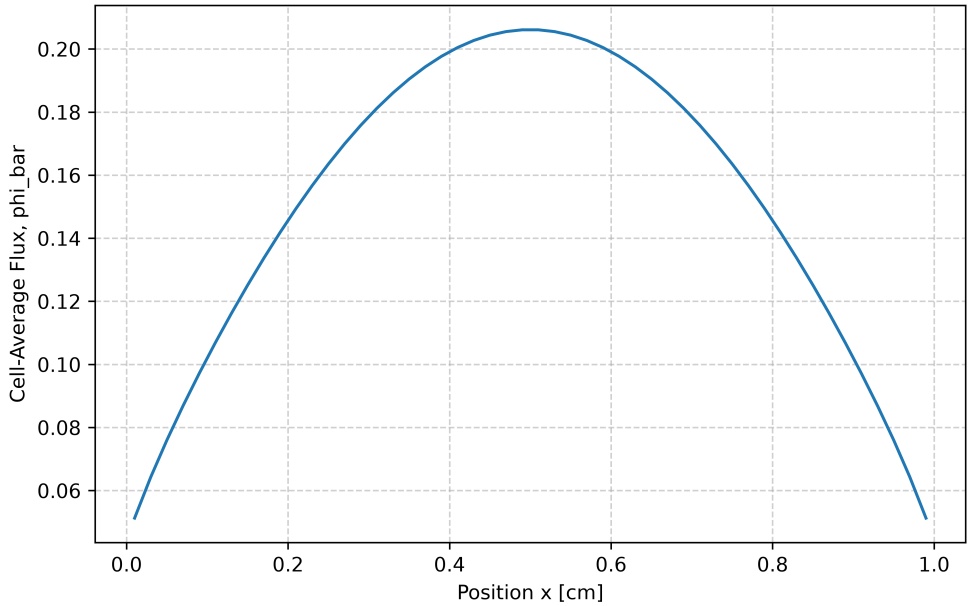


Figure 24: Cell-average scalar flux vs position.

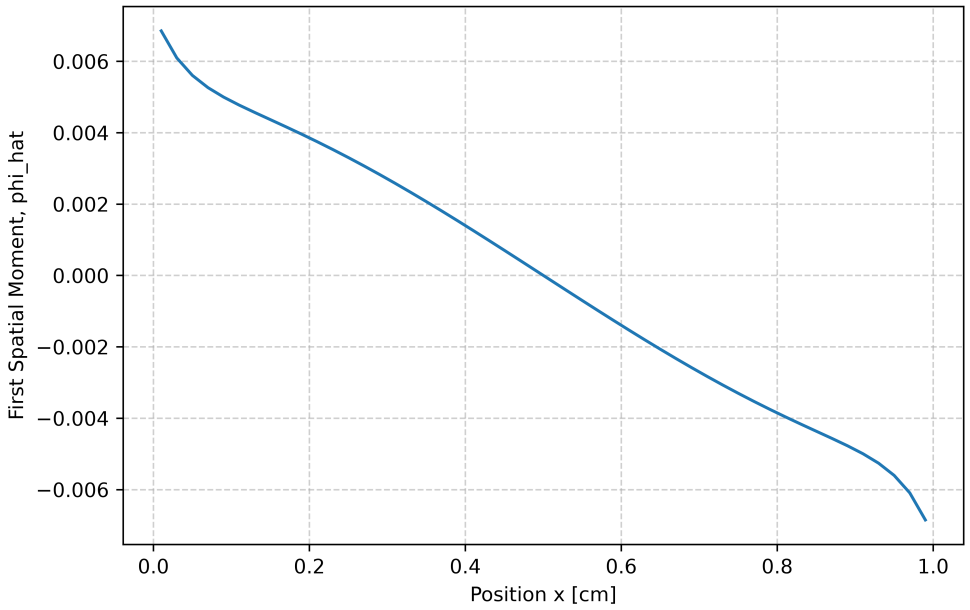


Figure 25: First spatial moment of the scalar flux vs position.

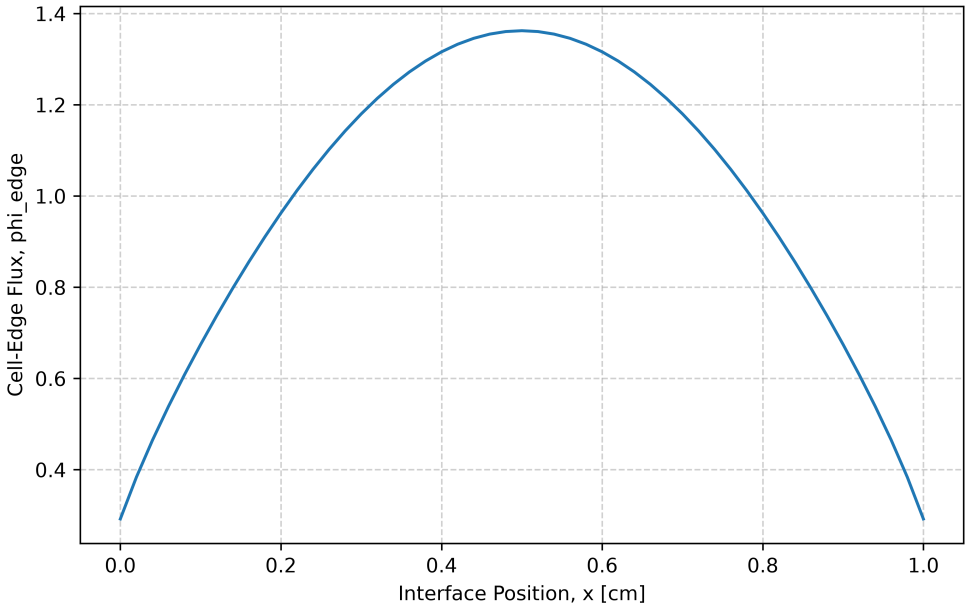


Figure 26: Cell-edge values of the scalar flux vs position.

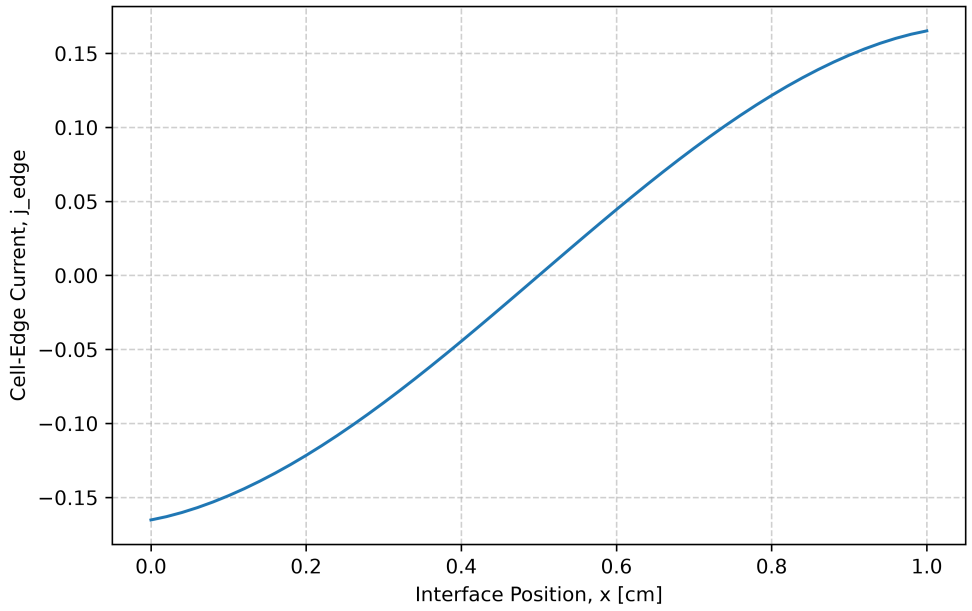


Figure 27: Cell-edge values of the current vs position.

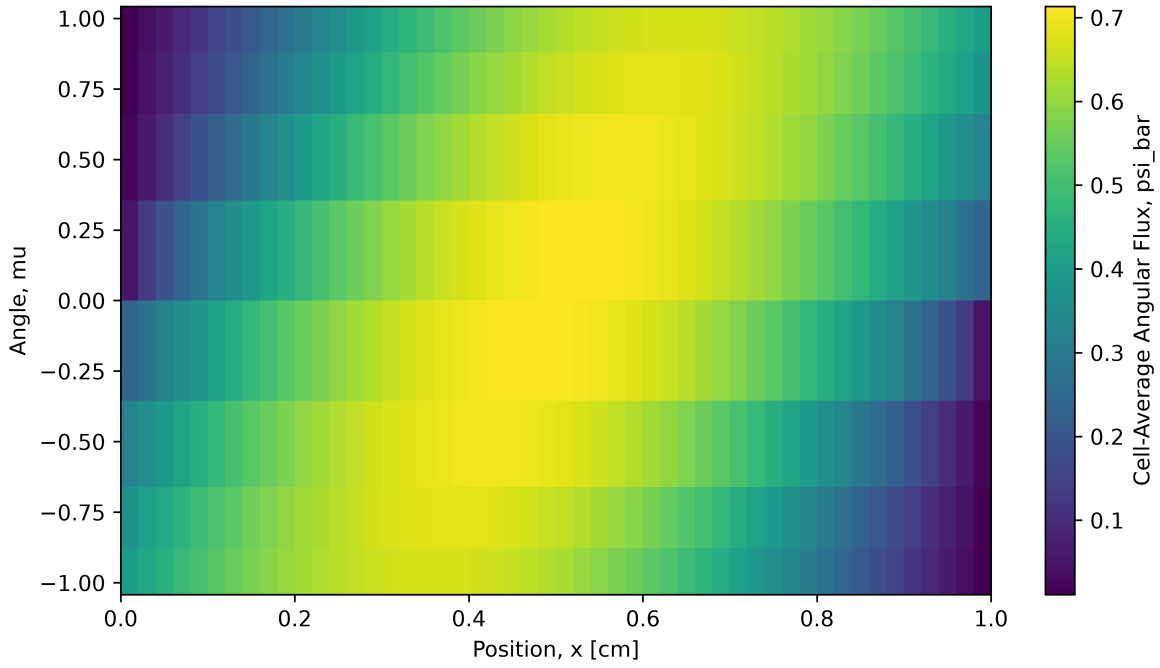


Figure 28: Cell-average values of the angular flux vs position and angle.

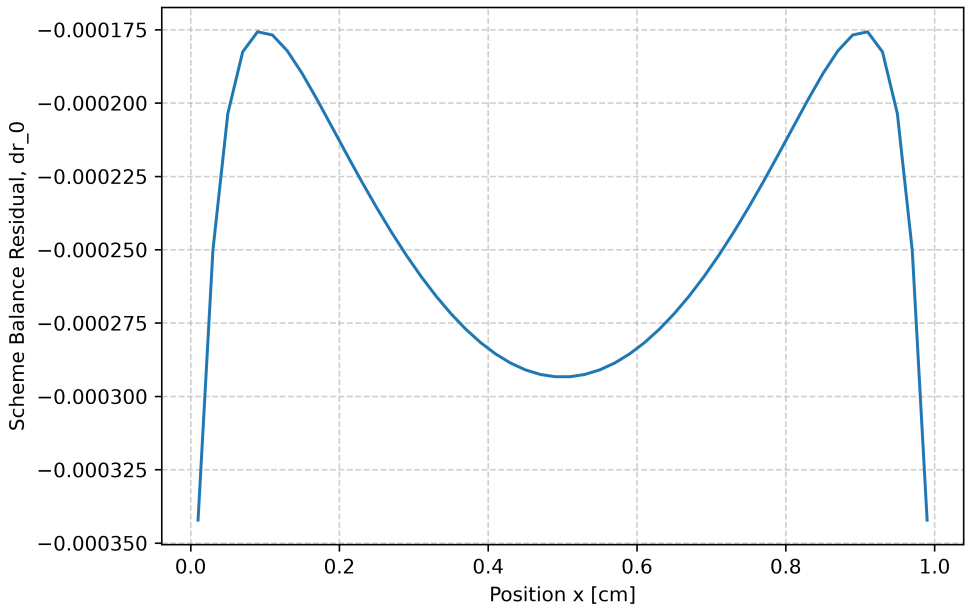


Figure 29: Scheme residual for the particle balance equation vs position.

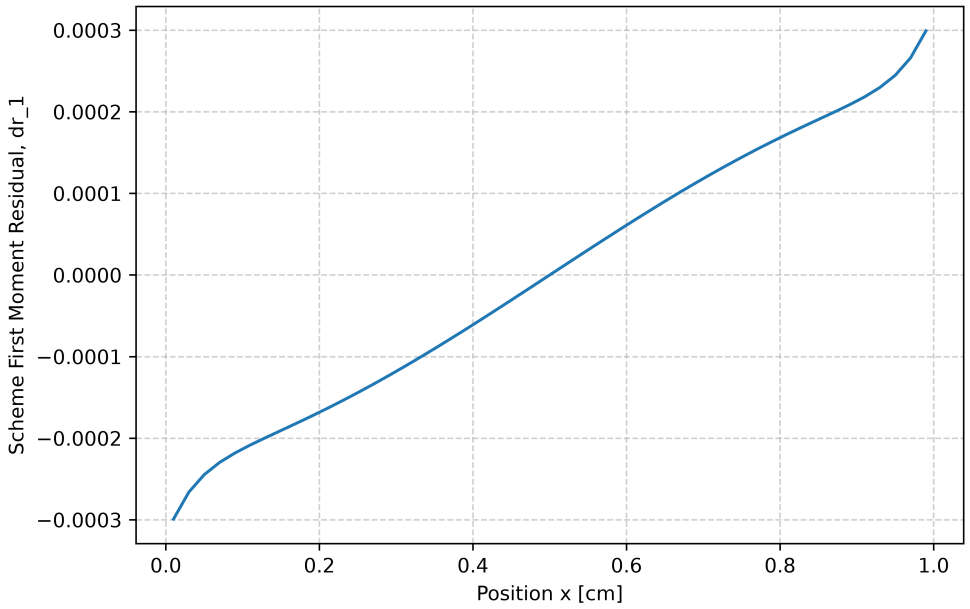


Figure 30: Scheme residual for the first moment of the particle balance equation vs position.

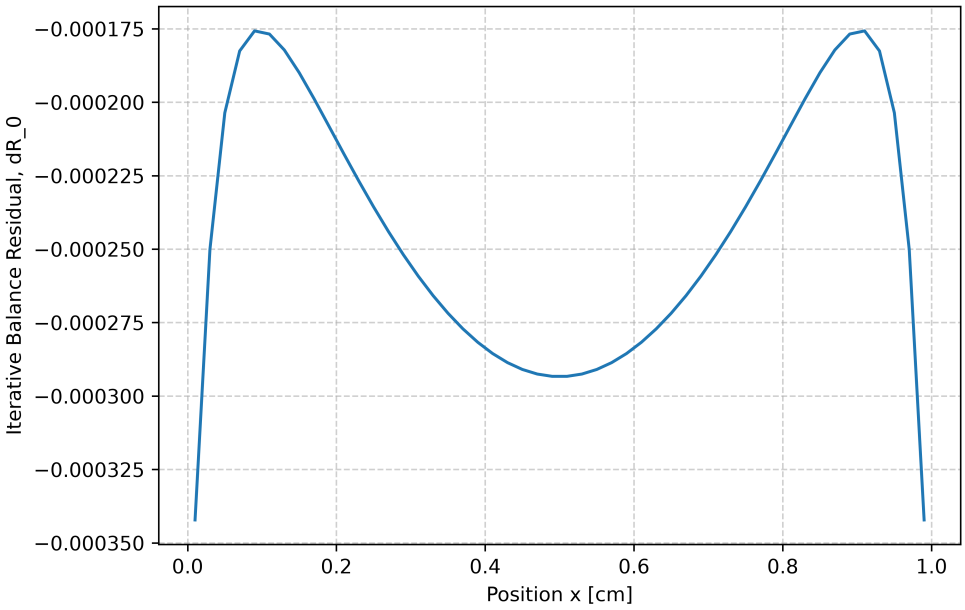


Figure 31: Iterative residual for the particle balance equation vs position.

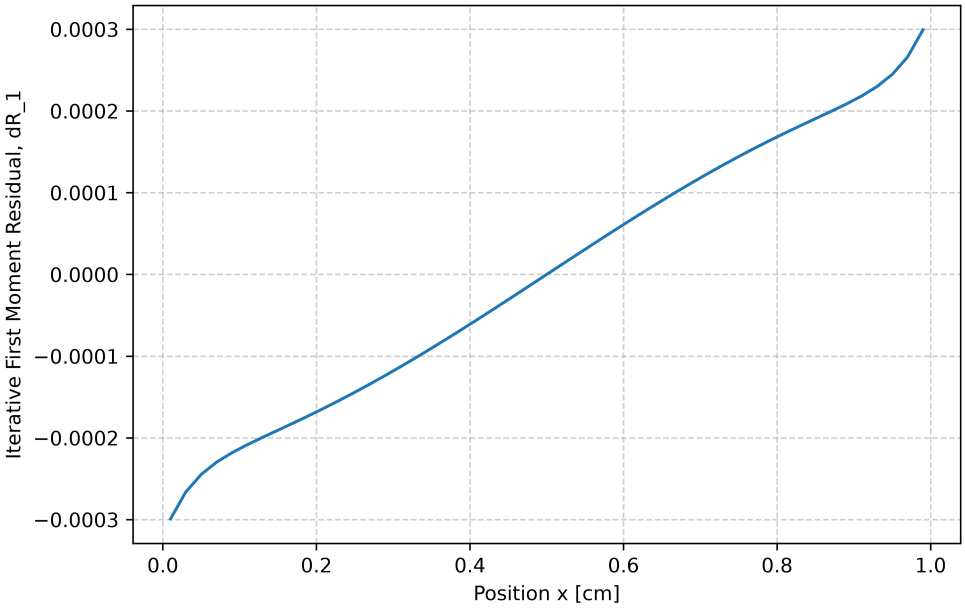


Figure 32: Iterative residual for the first moment of the particle balance equation vs position.

A Appendix for Python Code

Listing 1: test_a.py

```
# input parameters for "test_a"
problem_type = "fixed_source"
problem_name = "test_a"

convergence_criteria = 1e-6

uniform_dx = True
n_mesh_cells = 40
legendre_order = 8

left_bc  = "reflective"
right_bc = "vacuum"

region_boundaries = [0, 2, 3, 5, 6, 8]
sigma_ts = [50, 5, 0, 1.1, 1.1]
sigma_ss = [ 0, 0, 0, 0.7, 0.5]
sigma_fs = [ 0, 0, 0, 0.1, 0.16]

nu_fs = [ 0, 0, 0, 2, 2.5]
qs    = [50, 0, 0, 0, 1]
```

Listing 2: test_b.py

```
# input parameters for "test_b"
problem_type = "fixed_source"
problem_name = "test_b"

convergence_criteria = 1e-6

uniform_dx = False
n_mesh_cells = [10, 10]
legendre_order = 8

left_bc = "incident"
left_incident_flux = 1
right_bc = "vacuum"

region_boundaries = [0, 1, 11]
sigma_ts = [2, 100]
sigma_ss = [0, 100]
sigma_fs = [0,   0]

nu_fs = [0, 0]
qs    = [0, 0]
```

Listing 3: test_c.py

```
# input parameters for "test_c"
problem_type = "k-eigenvalue"
problem_name = "test_c"

convergence_criteria = 1e-6

uniform_dx = True
n_mesh_cells = 50
legendre_order = 8

left_bc  = "vacuum"
right_bc = "vacuum"

# the second region boundary is the initial guess for a
region_boundaries = [0, 1.0]
sigma_ts = [1]
sigma_ss = [0.45]
sigma_fs = [0.3]

nu_fs = [2]
qs    = [0]
```

Listing 4: main.py

```
# choose which problem by uncommenting the desired import
import test_a as problem
#import test_b as problem
#import test_c as problem

# !!! you can switch the mode of this file to run the plotting feature !!!

# CALCULATION MODE (calc)
#  - calc mode is the default mode
#  - running 'python main.py' executes uncommented test (imported as problem)

# PLOTTING MODE (plot)
#  - you can run plot mode by passing a command line argument
#  - 'python main.py --mode plot' will execute the script in the plotting mode
#  - plot mode is decoupled from the solution mode so you dont have to run an
#      hour long test (test b) to manipulate the results

# package imports
import numpy as np
from pathlib import Path
import time
import sys

# module imports
import mesh_builder
import output_manager
import plots
import quadrature
import solvers
```

```
# running the problem
def main():
    start_time = time.time()

    # execution mode settings
    mode = "calc"  # do calculation by default
    #mode = "plot"  # do plotting by default

    # check the command line arguments
    if len(sys.argv) > 1:
        for idx, arg in enumerate(sys.argv):
            # continue if there is a mode passed after '--mode'
            if arg == "—mode" and idx + 1 < len(sys.argv):
                mode = sys.argv[idx + 1].lower()

    # setting up results
    results_dir = Path(f"results-{problem.problem_name}")

    # safety checks for if the directory exists or not based on mode
    if mode == "plot":
        print(f"\n!!! [PLOTTING MODE] EXECUTING !!!")
        if results_dir.exists():
            print(f"\n    Generating plots in existing results dir '{results_dir}'")
            plots.generate_report_plots(results_dir)
            print(f"\n [PLOTTING MODE] COMPLETE !!!")
            return

        if not results_dir.exists():
            msg = f"Directory '{results_dir}' does not exist!
            Run the current case in 'calc' mode before executing 'plot' mode."
            raise FileNotFoundError(msg)
```

```
    if mode == "calc":
        if results_dir.exists():
            print(f"!!! WARNING !!! The directory '{results_dir}' already exists!")
            print("You will not be able to run in 'calc' mode until this
directorry has a different name.")
            print("Either rename or delete the directory")
            return

    if mode not in ["calc", "plot"]:
        msg = f"Unsupported mode '{mode}'. Supported modes = ['calc', 'plot']"
        raise ValueError(msg)

    results_dir.mkdir(parents=True, exist_ok=True)

    log_filename = results_dir / f"{problem.problem_name}.log"

    print(f"\n [CALCULATION MODE] BEGINNING")
    with open(log_filename, "w") as log_file:
        output_manager.print_problem_setup(problem, log_file)

        # discretize space and angle
        mesh_dict = mesh_builder.discretize_space(problem)
        quad_dict = quadrature.discretize_angle(problem)

        output_manager.log_time(
            start_time, f"Problem Setup Complete", file_obj=log_file
        )

        # fixed source problem
        if problem.problem_type == "fixed_source":
            msg = "\n  Beginning Inner Source Iteration Solver"
            output_manager.log_msg(msg, log_file)
```

```
    results_dict = solvers.fixed_source_solver(
        mesh_dict, quad_dict, problem, k_eff=1.0, file_obj=log_file
    )

    msg = f"Fixed source iterations complete"
    output_manager.log_time(start_time, msg, log_file)


# k-eigenvalue criticality search problem
if problem.problem_type == "k-eigenvalue":
    msg = "\n  Beginning Outer Criticality Search Solver"
    output_manager.log_msg(msg, log_file)

    results_dict = solvers.critical_width_solver(
        quad_dict, problem, file_obj=log_file
    )

    msg = f"Criticality search complete"
    output_manager.log_time(start_time, msg, log_file)


# catching other cases
if problem.problem_type not in ["fixed_source", "k-eigenvalue"]:
    msg = f"Unsupported problem type: {problem.problem_type}"
    raise ValueError(msg)

# output results
msg = "\n  Exporting results"
output_manager.log_msg(msg, log_file)

output_manager.save_results(
```

```
            results_dir, problem, mesh_dict, results_dict
        )

    print(f"\n [CALCULATION_MODE] COMPLETE")
    return


if __name__ == "__main__":
    main()
```

Listing 5: mesh_builder.py

```
import numpy as np



def discretize_space(problem):
    if problem.uniform_dx:
        width = problem.region_boundaries[-1] - problem.region_boundaries[0]
        n_cells = problem.n_mesh_cells
        dx_value = width / n_cells

        dx = np.full(n_cells, dx_value)
        cell_edges = np.linspace(
                problem.region_boundaries[0],
                problem.region_boundaries[-1],
                n_cells + 1)

    if not problem.uniform_dx:
        dx_list = []
        cell_edges_list = [problem.region_boundaries[0]]

        for region_idx in range(len(problem.n_mesh_cells)):
            left_bound = problem.region_boundaries[region_idx]
            right_bound = problem.region_boundaries[region_idx + 1]

            region_width = right_bound - left_bound
            n_region_cells = problem.n_mesh_cells[region_idx]

            region_dx = region_width / n_region_cells
            dx_list.extend([region_dx] * n_region_cells)

            for cell_idx in range(1, n_region_cells + 1):
                cell_edges_list.append(left_bound + cell_idx * region_dx)
```

```
    dx = np.array(dx_list)
    cell_edges = np.array(cell_edges_list)

n_problem_cells = len(dx)
cell_centers = 0.5 * (cell_edges[:-1] + cell_edges[1:])

sigma_t = np.zeros(n_problem_cells)
sigma_s = np.zeros(n_problem_cells)
sigma_f = np.zeros(n_problem_cells)
nu_f    = np.zeros(n_problem_cells)
q       = np.zeros(n_problem_cells)

for cell_idx in range(n_problem_cells):
    cell_center = cell_centers[cell_idx]

    for region_idx in range(len(problem.sigma_ts)):
        left_bound = problem.region_boundaries[region_idx]
        right_bound = problem.region_boundaries[region_idx + 1]

        if left_bound <= cell_center < right_bound:
            sigma_t[cell_idx] = problem.sigma_ts[region_idx]
            sigma_s[cell_idx] = problem.sigma_ss[region_idx]
            sigma_f[cell_idx] = problem.sigma_fs[region_idx]
            nu_f[cell_idx]    = problem.nu_fs[region_idx]
            q[cell_idx]       = problem.qs[region_idx]
            break

mesh_dict = {
    "dx": dx,
    "cell_edges": cell_edges,
    "sigma_t": sigma_t,
```

```
    "sigma_s": sigma_s,
    "sigma_f": sigma_f,
    "nu_f": nu_f,
    "q": q,
}

return mesh_dict
```

Listing 6: output_manager.py

```
from pathlib import Path
import numpy as np
import time


def log_msg(text, file_obj=None):
    print(text)
    if file_obj is not None: file_obj.write(text + "\n")


def log_time(start_time, message, file_obj=None):
    elapsed_time = time.time() - start_time
    full_message = f"{message} (Elapsed Time: {elapsed_time:.4f} seconds)"
    log_msg(full_message, file_obj)


def print_problem_setup(problem, file_obj=None):
    lines = [
        "=" * 50,
        f"INITIALIZING SOLVER FOR '{problem.problem_name}'",
        "=" * 50,
        f"  Problem Type:              {problem.problem_type}",
        f"  Convergence Criteria:      {problem.convergence_criteria}",
        f"  Uniform Spatial Mesh:      {problem.uniform_dx}",
        f"    Number of Mesh Cells:    {problem.n_mesh_cells}",
        f"  Legendre Order:            {problem.legendre_order}",
        f"  Left BC:                   {problem.left_bc}",
    ]

    if hasattr(problem, "left_incident_flux"):
        lines.append(f"    Left Incident Flux:      {problem.left_incident_flux}")
```

```
    lines.extend(
        [
            f"  Right BC:                  {problem.right_bc}",
            f"  Region Boundaries:         {problem.region_boundaries}",
            f"  Total Cross Sections:      {problem.sigma_ts}",
            f"  Scattering Cross Sections: {problem.sigma_ss}",
            f"  Fission Cross Sections:    {problem.sigma_fs}",
            f"  Nu-Fission:                {problem.nu_fs}",
            f"  External Source:           {problem.qs}",
            "=" * 50,
        ]
    )

    for line in lines:
        log_msg(line, file_obj)


def save_results(results_dir, problem, mesh_dict, results_dict):
    problem_name = problem.problem_name
    problem_type = problem.problem_type

    save_cell_centered_moments_and_residuals(results_dir, problem_name, mesh_dict,
    results_dict)
    save_cell_edge_flux_and_current(results_dir, problem_name, mesh_dict, results_dict)
    save_angular_flux_moments(results_dir, problem_name, mesh_dict, results_dict)
    save_cell_edge_angular_flux(results_dir, problem_name, mesh_dict, results_dict)

    save_history_results(results_dir, problem_name, results_dict, problem_type)


def save_cell_centered_moments_and_residuals(results_dir, problem_name, mesh_dict,
```

```
results_dict):
    dx = mesh_dict["dx"]
    cell_edges = mesh_dict["cell_edges"]
    n_cells = len(dx)

    phi_bar = results_dict["cell_average_scalar_flux"]
    phi_hat = results_dict["first_legendre_moment_scalar_flux"]
    j_bar = results_dict["cell_average_current"]
    j_hat = results_dict["first_legendre_moment_current"]
    dr_0 = results_dict["delta_r_0"]
    dr_1 = results_dict["delta_r_1"]
    dR_0 = results_dict["delta_R_0"]
    dR_1 = results_dict["delta_R_1"]

    out_file = results_dir / f"{problem_name}_cell_moments_and_residuals.txt"
    with open(out_file, "w") as f:
        f.write(f"CELL-CENTERED MOMENTS AND RESIDUALS for {problem_name}\n")
        f.write(
            f"{'Cell idx':<10s}{'x_i (cm)':<12s}"
            f"{'phi_bar_i':<14s}{'phi_hat_i':<14s}"
            f"{'j_bar_i':<14s}{'j_hat_i':<14s}"
            f"{'dr_0':<14s}{'dr_1':<14s}"
            f"{'dR_0':<14s}{'dR_1':<14s}\n"
        )

        for cell_idx in range(n_cells):
            cell_center = cell_edges[cell_idx] + 0.5 * dx[cell_idx]
            f.write(
                f"{cell_idx:<10d}{cell_center:<12.5f}"
                f"{phi_bar[cell_idx]:<14.6e}{phi_hat[cell_idx]:<14.6e}"
                f"{j_bar[cell_idx]:<14.6e}{j_hat[cell_idx]:<14.6e}"
                f"{dr_0[cell_idx]:<14.6e}{dr_1[cell_idx]:<14.6e}"
```

```
                f"{dR_0[cell_idx]:<14.6e}{dR_1[cell_idx]:<14.6e}\n"
            )


def save_cell_edge_flux_and_current(results_dir, problem_name, mesh_dict, results_dict):
    cell_edges = mesh_dict["cell_edges"]
    n_cells = len(mesh_dict["dx"])

    phi_edges = results_dict["cell_edge_scalar_flux"]
    j_edges = results_dict["cell_edge_current"]

    out_file = results_dir / f"{problem_name}_cell_edge_flux_and_current.txt"
    with open(out_file, "w") as f:
        f.write(f"CELL EDGE QUANTITIES for {problem_name}\n")
        f.write(
            f"{'Edge idx':<12s}{'x_i+1/2 (cm)':<15s}"
            f"{'phi_edge':<20s}{'j_edge':<20s}\n"
        )

        for edge_idx in range(n_cells + 1):
            f.write(
                f"{edge_idx:<12d}{cell_edges[edge_idx]:<15.5f}"
                f"{phi_edges[edge_idx]:<20.6e}{j_edges[edge_idx]:<20.6e}\n"
            )


def save_angular_flux_moments(results_dir, problem_name, mesh_dict, results_dict):
    dx = mesh_dict["dx"]
    cell_edges = mesh_dict["cell_edges"]
    n_cells = len(dx)

    psi_0moment = results_dict["cell_average_angular_flux"]
```

```
    psi_1moment = results_dict["first_legendre_moment_angular_flux"]
    n_angles = psi_0moment.shape[0]

    out_file = results_dir / f"{problem_name}_angular_flux_moments.txt"
    with open(out_file, "w") as f:
        f.write(f"2D ANGULAR FLUX MOMENTS FOR {problem_name}\n")
        f.write(f"Format for each entry: psi_bar / psi_hat\n")
        f.write(f"{'Angle idx':<12s}")
        for cell_idx in range(n_cells):
            x_i = cell_edges[cell_idx] + 0.5 * dx[cell_idx]
            f.write(f"| x={x_i:.3f}          ")
        f.write("\n" + "-" * (14 + 22 * n_cells) + "\n")

        for angle_idx in range(n_angles):
            f.write(f"Angle {angle_idx:<6d}")
            for cell_idx in range(n_cells):
                val_bar = psi_0moment[angle_idx, cell_idx]
                val_hat = psi_1moment[angle_idx, cell_idx]
                f.write(f"  {val_bar:.3e}/{val_hat:+.3e}")
            f.write("\n")


def save_cell_edge_angular_flux(results_dir, problem_name, mesh_dict, results_dict):
    cell_edges = mesh_dict["cell_edges"]
    n_cells = len(mesh_dict["dx"])

    psi_edge = results_dict["cell_edge_angular_flux"]
    n_angles = psi_edge.shape[0]

    out_file = results_dir / f"{problem_name}_cell_edge_angular_flux.txt"
    with open(out_file, "w") as f:
        f.write(f"2D CELL EDGE ANGULAR FLUX INTERFACES FOR {problem_name}\n")
```

```
        f.write(f"{'Angle␣idx':<12s}")
        for edge_idx in range(n_cells + 1):
            f.write(f"|␣x_e={cell_edges[edge_idx]:.3f}␣␣␣␣")
        f.write("\n" + "-" * (14 + 18 * (n_cells + 1)) + "\n")

        for angle_idx in range(n_angles):
            f.write(f"Angle␣{angle_idx:<6d}")
            for edge_idx in range(n_cells + 1):
                f.write(f"␣␣{psi_edge[angle_idx,␣edge_idx]:.4e}")
            f.write("\n")


def save_history_results(results_dir, problem_name, results_dict, problem_type):
    out_file = results_dir / f"{problem_name}_convergence_history.txt"

    with open(out_file, "w") as f:
        if problem_type == "fixed_source":
            history = results_dict["iteration_history"]
            f.write(f"INNER␣ITERATION␣CONVERGENCE␣HISTORY␣for␣{problem_name}\n")
            f.write(f"Total␣Source␣Iterations␣(N_iter)␣completed:␣{len(history)}\n")
            f.write(f"{'Iteration␣idx':<15s}{'Max-Norm␣Error':<25s}
␣␣␣␣␣␣␣␣␣␣␣{'Spectral␣Radius␣(rho)':<25s}\n")

            for row_idx in range(len(history)):
                row_data = history[row_idx]
                rho_val = row_data["rho"]
                rho_str = f"{rho_val:.5f}" if rho_val is not None else "None"
                f.write(f"{row_data['iteration']:<15d}{row_data['error']:<25.5e}
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣{rho_str:<25s}\n")

        if problem_type == "k-eigenvalue":
            inner_history = results_dict["inner_history"]
```

```
            outer_history = results_dict["outer_history"]

            f.write(f"CRITICALITY SEARCH ITERATION HISTORIES LOG for {problem_name}\n\n")
            f.write("1. Outer loop: secant method geometry search widths\n")
            f.write(f"{'Outer Step idx':<18s}{'Slab Width a_crit (cm)':<25s}\n")

            for outer_idx in range(len(outer_history)):
                outer_row = outer_history[outer_idx]
                f.write(f"{outer_row['iteration']:<18d}{outer_row['a_crit']:<25.5f}\n")

            f.write("\n2. Inner power iteration balance track (from final iteration)\n")
            f.write(f"{'Inner Step idx':<16s}{'Error (phi)':<20s}{'Error (k)':<20s}
            {'k_effective':<18s}\n")

            for inner_idx in range(len(inner_history)):
                inner_row = inner_history[inner_idx]
                f.write(
                    f"{inner_row['iteration']:<16d}"
                    f"{inner_row['error_phi']:<20.5e}"
                    f"{inner_row['error_k']:<20.5e}"
                    f"{inner_row['k_eff']:<18.6f}\n"
                )
```

Listing 7: plots.py

```
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path


def generate_report_plots(results_dir):
    problem_name = results_dir.name.replace("results-", "")
    history_file = results_dir / f"{problem_name}_convergence_history.txt"

    plot_dir = results_dir / "plots"
    plot_dir.mkdir(parents=True, exist_ok=True)
    print(f"\nSavings plots to '{plot_dir}'")

    with open(history_file, "r") as f:
        header = f.readline()

    plot_common_results(results_dir, plot_dir, problem_name)

    if "INNER ITERATION" in header:
        plot_fixed_source_convergence(results_dir, plot_dir, problem_name, history_file)
    if "CRITICALITY SEARCH" in header:
        plot_critical_width_history(results_dir, plot_dir, problem_name, history_file)

    return


def plot_common_results(results_dir, plot_dir, problem_name):
    moments_file = results_dir / f"{problem_name}_cell_moments_and_residuals.txt"
    edges_file = results_dir / f"{problem_name}_cell_edge_flux_and_current.txt"
    angular_moments_file = results_dir / f"{problem_name}_angular_flux_moments.txt"
    angular_edge_file = results_dir / f"{problem_name}_cell_edge_angular_flux.txt"
```

```
if moments_file.exists():
    data = np.loadtxt(moments_file, skiprows=2)
    x_centers = data[:, 1]

    plot_cell_average_flux(results_dir, plot_dir, problem_name, x_centers,
    data[:, 2])
    plot_spatial_moment_flux(results_dir, plot_dir, problem_name, x_centers,
    data[:, 3])
    plot_cell_average_current(results_dir, plot_dir, problem_name, x_centers,
    data[:, 4])
    plot_spatial_moment_current(results_dir, plot_dir, problem_name, x_centers,
    data[:, 5])
    plot_residual_dr0(results_dir, plot_dir, problem_name, x_centers, data[:, 6])
    plot_residual_dr1(results_dir, plot_dir, problem_name, x_centers, data[:, 7])
    plot_residual_dR0(results_dir, plot_dir, problem_name, x_centers, data[:, 8])
    plot_residual_dR1(results_dir, plot_dir, problem_name, x_centers, data[:, 9])

if edges_file.exists():
    data = np.loadtxt(edges_file, skiprows=2)
    x_edges = data[:, 1]
    plot_cell_edge_flux(results_dir, plot_dir, problem_name, x_edges, data[:, 2])
    plot_cell_edge_current(results_dir, plot_dir, problem_name, x_edges, data[:, 3])

if angular_moments_file.exists():
    plot_angular_moments_2d(results_dir, plot_dir, problem_name,
    angular_moments_file)

if angular_edge_file.exists():
    plot_angular_edge_2d(results_dir, plot_dir, problem_name, angular_edge_file)
```

```
def plot_cell_average_flux(results_dir, plot_dir, problem_name, x_centers, phi_bar):
    plt.figure(figsize=(7, 4.5))
    plt.plot(x_centers, phi_bar)
    plt.xlabel("Position x [cm]")
    plt.ylabel("Cell-Average Flux, phi_bar")
    plt.grid(True, linestyle='--', alpha=0.6)
    plt.tight_layout()
    plt.savefig(plot_dir / f"{problem_name}_cell_average_flux.png", dpi=600)
    plt.close()


def plot_spatial_moment_flux(results_dir, plot_dir, problem_name, x_centers, phi_hat):
    plt.figure(figsize=(7, 4.5))
    plt.plot(x_centers, phi_hat)
    plt.xlabel("Position x [cm]")
    plt.ylabel("First Spatial Moment, phi_hat")
    plt.grid(True, linestyle='--', alpha=0.6)
    plt.tight_layout()
    plt.savefig(plot_dir / f"{problem_name}_spatial_moment_flux.png", dpi=600)
    plt.close()


def plot_cell_average_current(results_dir, plot_dir, problem_name, x_centers, j_bar):
    plt.figure(figsize=(7, 4.5))
    plt.plot(x_centers, j_bar)
    plt.xlabel("Position x [cm]")
    plt.ylabel("Cell-Average Current, j_bar")
    plt.grid(True, linestyle='--', alpha=0.6)
    plt.tight_layout()
    plt.savefig(plot_dir / f"{problem_name}_cell_average_current.png", dpi=600)
    plt.close()
```

```
def plot_spatial_moment_current(results_dir, plot_dir, problem_name, x_centers, j_hat):
    plt.figure(figsize=(7, 4.5))
    plt.plot(x_centers, j_hat)
    plt.xlabel("Position x [cm]")
    plt.ylabel("First Spatial Moment of Current, j_hat")
    plt.grid(True, linestyle='—', alpha=0.6)
    plt.tight_layout()
    plt.savefig(plot_dir / f"{problem_name}_spatial_moment_current.png", dpi=600)
    plt.close()


def plot_residual_dr0(results_dir, plot_dir, problem_name, x_centers, dr_0):
    plt.figure(figsize=(7, 4.5))
    plt.plot(x_centers, dr_0)
    plt.xlabel("Position x [cm]")
    plt.ylabel("Scheme Balance Residual, dr_0")
    plt.grid(True, linestyle='—', alpha=0.6)
    plt.tight_layout()
    plt.savefig(plot_dir / f"{problem_name}_residual_dr0.png", dpi=600)
    plt.close()


def plot_residual_dr1(results_dir, plot_dir, problem_name, x_centers, dr_1):
    plt.figure(figsize=(7, 4.5))
    plt.plot(x_centers, dr_1)
    plt.xlabel("Position x [cm]")
    plt.ylabel("Scheme First Moment Residual, dr_1")
    plt.grid(True, linestyle='—', alpha=0.6)
    plt.tight_layout()
    plt.savefig(plot_dir / f"{problem_name}_residual_dr1.png", dpi=600)
    plt.close()
```

```
def plot_residual_dR0(results_dir, plot_dir, problem_name, x_centers, dR_0):
    plt.figure(figsize=(7, 4.5))
    plt.plot(x_centers, dR_0)
    plt.xlabel("Position x [cm]")
    plt.ylabel("Iterative Balance Residual, dR_0")
    plt.grid(True, linestyle='—', alpha=0.6)
    plt.tight_layout()
    plt.savefig(plot_dir / f"{problem_name}_residual_dR0.png", dpi=600)
    plt.close()


def plot_residual_dR1(results_dir, plot_dir, problem_name, x_centers, dR_1):
    plt.figure(figsize=(7, 4.5))
    plt.plot(x_centers, dR_1)
    plt.xlabel("Position x [cm]")
    plt.ylabel("Iterative First Moment Residual, dR_1")
    plt.grid(True, linestyle='—', alpha=0.6)
    plt.tight_layout()
    plt.savefig(plot_dir / f"{problem_name}_residual_dR1.png", dpi=600)
    plt.close()


def plot_cell_edge_flux(results_dir, plot_dir, problem_name, x_edges, phi_edges):
    plt.figure(figsize=(7, 4.5))
    plt.plot(x_edges, phi_edges)
    plt.xlabel("Interface Position, x [cm]")
    plt.ylabel("Cell–Edge Flux, phi_edge")
    plt.grid(True, linestyle='—', alpha=0.6)
    plt.tight_layout()
    plt.savefig(plot_dir / f"{problem_name}_cell_edge_flux.png", dpi=600)
```

```
    plt.close()


def plot_cell_edge_current(results_dir, plot_dir, problem_name, x_edges, j_edges):
    plt.figure(figsize=(7, 4.5))
    plt.plot(x_edges, j_edges)
    plt.xlabel("Interface Position, x [cm]")
    plt.ylabel("Cell-Edge Current, j_edge")
    plt.grid(True, linestyle='--', alpha=0.6)
    plt.tight_layout()
    plt.savefig(plot_dir / f"{problem_name}_cell_edge_current.png", dpi=600)
    plt.close()


def plot_angular_moments_2d(results_dir, plot_dir, problem_name, file_path):
    with open(file_path, 'r') as f:
        lines = f.readlines()

    pos_line = lines[2].strip()
    cell_segments = [x.strip() for x in pos_line.split('|')[1:] if
    x.strip() and '=' in x]
    cell_positions = [float(x.split('=')[1].strip()) for x in cell_segments]

    matrix_data_bar = []
    matrix_data_hat = []
    for line in lines[4:]:
        if "Angle" in line:
            parts = line.split()
            vals_bar = [float(p.split('/')[0]) for p in parts[2:]]
            vals_hat = [float(p.split('/')[1]) for p in parts[2:]]
            matrix_data_bar.append(vals_bar)
            matrix_data_hat.append(vals_hat)
```

```
    psi_bar = np.array(matrix_data_bar)
    psi_hat = np.array(matrix_data_hat)
    n_angles = psi_bar.shape[0]
    mu, _ = np.polynomial.legendre.leggauss(n_angles)

    X, Y = np.meshgrid(cell_positions, mu)

    plt.figure(figsize=(8, 4.5))
    plt.pcolormesh(X, Y, psi_bar, cmap='viridis', shading='nearest')
    plt.colorbar().set_label("Cell-Average Angular Flux, psi_bar")
    plt.xlabel("Position, x [cm]")
    plt.ylabel("Angle, mu")
    plt.tight_layout()
    plt.savefig(plot_dir / f"{problem_name}_angular_flux_average_2d.png", dpi=600)
    plt.close()

    plt.figure(figsize=(8, 4.5))
    plt.pcolormesh(X, Y, psi_hat, cmap='coolwarm', shading='nearest')
    plt.colorbar().set_label("First Spatial Moment Angular Flux, psi_hat")
    plt.xlabel("Position, x [cm]")
    plt.ylabel("Angle, mu")
    plt.tight_layout()
    plt.savefig(plot_dir / f"{problem_name}_angular_flux_moment_2d.png", dpi=600)
    plt.close()


def plot_angular_edge_2d(results_dir, plot_dir, problem_name, file_path):
    with open(file_path, 'r') as f:
        lines = f.readlines()

    pos_line = lines[1].strip()
```

```
    edge_segments = [x.strip() for x in pos_line.split('|')[1:] if
    x.strip() and '=' in x]
    edge_positions = [float(x.split('=')[1].strip()) for x in edge_segments]

    matrix_data = []
    for line in lines[3:]:
        if "Angle" in line:
            parts = line.split()
            matrix_data.append([float(val) for val in parts[2:]])

    psi_edge = np.array(matrix_data)
    n_angles = psi_edge.shape[0]
    mu, _ = np.polynomial.legendre.leggauss(n_angles)

    X, Y = np.meshgrid(edge_positions, mu)

    plt.figure(figsize=(8, 4.5))
    plt.pcolormesh(X, Y, psi_edge, cmap='viridis', shading='nearest')
    plt.colorbar().set_label("Cell-Edge Angular Flux, psi_edge")
    plt.xlabel("Interface Position, x [cm]")
    plt.ylabel("Angle, mu")
    plt.tight_layout()
    plt.savefig(plot_dir / f"{problem_name}_cell_edge_angular_flux_2d.png", dpi=600)
    plt.close()


def plot_fixed_source_convergence(results_dir, plot_dir, problem_name, history_file):
    raw_data = np.loadtxt(
        history_file,
        skiprows=3,
        converters={2: lambda s: float(s.replace('None', '0.0'))}
    )
```

```
    sampled_data = raw_data[::1000]

    if raw_data[-1, 0] != sampled_data[-1, 0]:
        sampled_data = np.vstack([sampled_data, raw_data[-1]])

    iters = sampled_data[:, 0]
    errors = sampled_data[:, 1]
    rhos = sampled_data[:, 2]

    plt.figure(figsize=(7, 4.5))
    plt.semilogy(iters, errors)
    plt.xlabel("Iteration index")
    plt.ylabel("L-infinity Error")
    plt.grid(True, which="both", linestyle='--', alpha=0.5)
    plt.tight_layout()
    plt.savefig(plot_dir / f"{problem_name}_error_history.png", dpi=600)
    plt.close()

    plt.figure(figsize=(7, 4.5))
    plt.plot(iters, rhos)
    plt.xlabel("Iteration index")
    plt.ylabel("Spectral Radius")
    plt.grid(True, linestyle='--', alpha=0.6)
    plt.tight_layout()
    plt.savefig(plot_dir / f"{problem_name}_spectral_radius_history.png", dpi=600)
    plt.close()


def plot_critical_width_history(results_dir, plot_dir, problem_name, history_file):
    outer_lines = []
    with open(history_file, 'r') as f:
        capture = False
```

```
    for line in f:
        if "1. Outer loop" in line:
            capture = True
            continue
        if "2. Inner power" in line:
            break
        if capture and line.strip() and not line.startswith("Outer Step"):
            outer_lines.append(line)

outer_iters = []
outer_widths = []
for line in outer_lines:
    parts = line.split()
    if len(parts) >= 2:
        try:
            outer_iters.append(int(parts[0]))
            outer_widths.append(float(parts[1]))
        except ValueError:
            continue

plt.figure(figsize=(7, 4.5))
plt.plot(outer_iters, outer_widths)
plt.xlabel("Secant Search Step Index")
plt.ylabel("Estimated Slab Width, a_crit [cm]")
plt.grid(True, linestyle='—', alpha=0.6)
plt.tight_layout()
plt.savefig(plot_dir / f"{problem_name}_critical_width_history.png", dpi=600)
plt.close()
```

Listing 8: quadrature.py

```
import numpy as np



def discretize_angle(problem):
    order = problem.legendre_order

    mu, w = np.polynomial.legendre.leggauss(order)

    quad_dict = {
        "mu": mu,
        "w": w,
    }

    return quad_dict
```

Listing 9: solvers.py

```
import numpy as np
import time

import mesh_builder
import output_manager



def fixed_source_solver(mesh_dict, quad_dict, problem, k_eff=1.0, file_obj=None):
    """used to solve tests a and b"""
    epsilon = problem.convergence_criteria

    dx = mesh_dict["dx"]
    cell_edges = mesh_dict["cell_edges"]
    sigma_t = mesh_dict["sigma_t"]
    sigma_s = mesh_dict["sigma_s"]
    sigma_f = mesh_dict["sigma_f"]
    nu_f = mesh_dict["nu_f"]
    q = mesh_dict["q"]

    n_cells = len(dx)
    phi_L = np.zeros(n_cells)
    phi_R = np.zeros(n_cells)

    converged = False
    error_previous = 0.0
    iteration_idx = 0
    iteration_history = []

    psi_left_reflective = None
```

```
# precompute static "matrices"
# streaming & BC terms are constants

# collision
sigma_t_dx_over_3 = (1 / 3) * sigma_t * dx
sigma_t_dx_over_6 = (1 / 6) * sigma_t * dx

# scattering and fission source
production_coeff = sigma_s + (sigma_f * nu_f / k_eff)
internal_source_diag = (1 / 6) * production_coeff * dx
internal_source_off = (1 / 12) * production_coeff * dx

# external source
external_source = 0.25 * q * dx

# residual placeholders
delta_r_0, delta_r_1, delta_R_0, delta_R_1 = None, None, None, None

loop_start_time = time.time()

while not converged:
    iteration_idx += 1

    phi_L_prev = phi_L.copy()
    phi_R_prev = phi_R.copy()

    b_source_L = (internal_source_diag * phi_L_prev +
    internal_source_off * phi_R_prev) + external_source
    b_source_R = (internal_source_off * phi_L_prev +
    internal_source_diag * phi_R_prev) + external_source

    phi_L, phi_R, j_L, j_R, psi_left_reflective, psi_0moment_matrix,
```

```
psi_1moment_matrix, psi_edge_matrix = angular_mesh_sweep(
    mesh_dict, quad_dict,
    sigma_t_dx_over_3, sigma_t_dx_over_6,
    b_source_L, b_source_R,
    problem, psi_left_reflective
)

error_L = np.max(np.abs(phi_L - phi_L_prev))
error_R = np.max(np.abs(phi_R - phi_R_prev))
error_current = max(error_L, error_R)

rho = None

if iteration_idx >= 2 and error_previous > 0.0:
    rho = error_current / error_previous

    if 0.0 < rho < 1.0:
        threshold = epsilon * ((1 / rho) - 1.0)
        if error_current < threshold:
            converged = True

# calculate residuals once converged
if converged:
    phi_0moment_current = 0.5 * (phi_L + phi_R)
    phi_1moment_current = 0.5 * (phi_R - phi_L)

    phi_0moment_prev = 0.5 * (phi_L_prev + phi_R_prev)
    phi_1moment_prev = 0.5 * (phi_R_prev - phi_L_prev)

    q_0moment_integrated = q * dx
    q_1moment_integrated = np.zeros(n_cells)
```

```
            production = sigma_s + (sigma_f * nu_f / 1)

            # scheme residual, particle balance (Eq. 8a)
            delta_r_0 = (j_R - j_L) + (sigma_t * phi_0moment_current * dx) -
            (production * phi_0moment_prev * dx) - q_0moment_integrated

            # scheme residual, first moment (Eq. 8b)
            j_0moment = 0.5 * (j_L + j_R)
            delta_r_1 = 3.0 * (j_R + j_L - 2 * j_0moment) +
            (sigma_t * phi_1moment_current * dx) - (production * phi_1moment_prev * dx)
            - q_1moment_integrated

            # iterative residual, particle palance (Eq. 9a)
            delta_R_0 = (j_R - j_L) + ((sigma_t - production) * phi_0moment_current * dx)
            - q_0moment_integrated

            # iterative residual, first moment (Eq. 9b)
            delta_R_1 = 3.0 * (j_R + j_L - 2 * j_0moment) +
            ((sigma_t - production) * phi_1moment_current * dx) -
            q_1moment_integrated



        iteration_history.append({
            "iteration": iteration_idx,
            "error": error_current,
            "rho": rho,
        })

        if iteration_idx % 10 == 0:
            elapsed = time.time() - loop_start_time
            msg = f"  >  Iteration  {iteration_idx:<5d}  |  Current  Error:
  {error_current:.4e}  |  Elapsed  Time:  {elapsed:.3f}s"
```

```
            output_manager.log_msg(msg, file_obj)

        error_previous = error_current

    phi_ie_edge = np.zeros(n_cells + 1)
    j_ie_edge = np.zeros(n_cells + 1)
    for edge_idx in range(n_cells + 1):
        phi_ie_edge[edge_idx] = np.sum(quad_dict["w"] * psi_edge_matrix[:, edge_idx])
        j_ie_edge[edge_idx] = np.sum(quad_dict["w"] * quad_dict["mu"] *
        psi_edge_matrix[:, edge_idx])

    results_dict = {
        "cell_average_angular_flux": psi_0moment_matrix,
        "cell_edge_angular_flux": psi_edge_matrix,
        "first_legendre_moment_angular_flux": psi_1moment_matrix,
        "cell_average_scalar_flux": 0.5 * (phi_R + phi_L),
        "cell_edge_scalar_flux": phi_ie_edge,
        "first_legendre_moment_scalar_flux": 0.5 * (phi_R - phi_L),
        "cell_average_current": 0.5 * (j_R + j_L),
        "cell_edge_current": j_ie_edge,
        "first_legendre_moment_current": 0.5 * (j_R - j_L),
        "phi_L": phi_L,
        "phi_R": phi_R,
        "j_L": j_L,
        "j_R": j_R,
        "iteration_history": iteration_history,
        "delta_r_0": delta_r_0,
        "delta_r_1": delta_r_1,
        "delta_R_0": delta_R_0,
        "delta_R_1": delta_R_1,}
    return results_dict
```

```
def angular_mesh_sweep(
    mesh_dict, quad_dict,
    sigma_t_dx_over_3, sigma_t_dx_over_6,
    b_source_L, b_source_R,
    problem, psi_left_reflective
):
    dx = mesh_dict["dx"]
    mu_arr = quad_dict["mu"]
    w_arr = quad_dict["w"]
    n_cells = len(dx)
    n_angles = len(mu_arr)

    phi_L_new = np.zeros(n_cells)
    phi_R_new = np.zeros(n_cells)
    j_L_new = np.zeros(n_cells)
    j_R_new = np.zeros(n_cells)

    psi_0moment_matrix = np.zeros((n_angles, n_cells))
    psi_1moment_matrix = np.zeros((n_angles, n_cells))
    psi_edge_matrix = np.zeros((n_angles, n_cells + 1))

    if psi_left_reflective is None:
        psi_left_reflective = np.zeros(n_angles)

    for m_idx, mu in enumerate(mu_arr):
        w = w_arr[m_idx]

        # sweep to the right (forward)
        if mu > 0:
            if problem.left_bc == "incident":
                psi_incoming = problem.left_incident_flux
```

```
if problem.left_bc == "reflective":
    psi_incoming = psi_left_reflective[m_idx]
if problem.left_bc == "vacuum":
    psi_incoming = 0.0

psi_edge_matrix[m_idx, 0] = psi_incoming

for cell_idx in range(n_cells):
    # streaming w/o incoming flux + total loss
    a11 = 0.5 * mu + sigma_t_dx_over_3[cell_idx]
    a12 = 0.5 * mu + sigma_t_dx_over_6[cell_idx]
    a21 = -0.5 * mu + sigma_t_dx_over_6[cell_idx]
    a22 = 0.5 * mu + sigma_t_dx_over_3[cell_idx]

    # sources + incoming flux
    b1 = b_source_L[cell_idx] + mu * psi_incoming
    b2 = b_source_R[cell_idx]

    # calc det for analytical inversion
    det = a11 * a22 - a12 * a21
    psi_L = (b1 * a22 - b2 * a12) / det
    psi_R = (a11 * b2 - a21 * b1) / det

    psi_0moment_matrix[m_idx, cell_idx] = 0.5 * (psi_L + psi_R)
    psi_1moment_matrix[m_idx, cell_idx] = 0.5 * (psi_R - psi_L)
    psi_edge_matrix[m_idx, cell_idx + 1] = psi_R

    # add flux contribution
    phi_L_new[cell_idx] += w * psi_L
    phi_R_new[cell_idx] += w * psi_R
    j_L_new[cell_idx] += w * mu * psi_L
    j_R_new[cell_idx] += w * mu * psi_R
```

```
            # set incoming flux for next cell
            psi_incoming = psi_R
        continue

    # sweep to the left (backward)
    if mu < 0:
        if problem.right_bc == "vacuum":
            psi_incoming = 0.0
        if problem.right_bc != "vacuum":
            raise ValueError("Support for non-vacuum BC not added for right")

        psi_edge_matrix[m_idx, n_cells] = psi_incoming

        for cell_idx in range(n_cells - 1, -1, -1):
            # streaming w/o incoming flux + total loss
            a11 = -0.5 * mu + sigma_t_dx_over_3[cell_idx]
            a12 = 0.5 * mu + sigma_t_dx_over_6[cell_idx]
            a21 = -0.5 * mu + sigma_t_dx_over_6[cell_idx]
            a22 = -0.5 * mu + sigma_t_dx_over_3[cell_idx]

            # sources - incoming flux
            b1 = b_source_L[cell_idx]
            b2 = b_source_R[cell_idx] - mu * psi_incoming

            # calc det for analytical inversion
            det = a11 * a22 - a12 * a21
            psi_L = (b1 * a22 - b2 * a12) / det
            psi_R = (a11 * b2 - a21 * b1) / det

            psi_0moment_matrix[m_idx, cell_idx] = 0.5 * (psi_L + psi_R)
            psi_1moment_matrix[m_idx, cell_idx] = 0.5 * (psi_R - psi_L)
```

```
                psi_edge_matrix[m_idx, cell_idx] = psi_L

                # add flux contribution
                phi_L_new[cell_idx] += w * psi_L
                phi_R_new[cell_idx] += w * psi_R
                j_L_new[cell_idx] += w * mu * psi_L
                j_R_new[cell_idx] += w * mu * psi_R

                # set incoming flux for next cell
                psi_incoming = psi_L

            if problem.left_bc == "reflective":
                mirrored_idx = n_angles - m_idx - 1
                psi_left_reflective[mirrored_idx] = psi_L

            continue

    return phi_L_new, phi_R_new, j_L_new, j_R_new, psi_left_reflective,
    psi_0moment_matrix, psi_1moment_matrix, psi_edge_matrix


def critical_width_solver(quad_dict, problem, file_obj=None):
    """used to solve test c"""
    outer_history = []

    a_0 = problem.region_boundaries[-1]
    a_1 = a_0 * 1.15

    k_0, _, _, _, _, _, _, _, _, _, _, _, _ = eigenvalue_inner_loop(
    a_0, quad_dict, problem, file_obj)
    msg = f"Initial Guess 1: width = {a_0:.4f} cm; k_eff = {k_0:.6f}"
    output_manager.log_msg(msg, file_obj)
```

```
    k_1, _, _, _, _, _, _, _, _, _, _, _, _ = eigenvalue_inner_loop(
    a_1, quad_dict, problem, file_obj)
    msg = f"Initial Guess 2: width = {a_1:.4f} cm; k_eff = {k_1:.6f}"
    output_manager.log_msg(msg, file_obj)

    outer_converged = False
    outer_idx = 0
    max_outer_iters = 50

    phi_L, phi_R, j_L, j_R, history = None, None, None, None, None
    psi_0moment_matrix, psi_1moment_matrix, psi_edge_matrix = None, None, None
    delta_r_0, delta_r_1, delta_R_0, delta_R_1 = None, None, None, None

    while not outer_converged and outer_idx < max_outer_iters:
        outer_idx += 1

        a_next = a_1 - (k_1 - 1.0) * (a_1 - a_0) / (k_1 - k_0)

        (k_next, phi_L, phi_R, j_L, j_R, history,
         psi_0moment_matrix, psi_1moment_matrix, psi_edge_matrix,
         delta_r_0, delta_r_1, delta_R_0, delta_R_1) = eigenvalue_inner_loop(
         a_next, quad_dict, problem, file_obj)

        outer_history.append({"iteration": outer_idx, "a_crit": a_next})

        msg = f"Outer Step {outer_idx:<2d}: width = {a_next:.5f} cm; k_eff
        = {k_next:.6f}"
        output_manager.log_msg(msg, file_obj)

        if abs(k_next - 1.0) < 1e-5:
            msg = f"!!! Critical width found (k=1), a = {a_next:.5f} cm\n !!!"
```

```
            output_manager.log_msg(msg, file_obj)
            outer_converged = True

        a_0 = a_1
        a_1 = a_next
        k_0 = k_1
        k_1 = k_next

n_cells = len(phi_L)
phi_ie_edge = np.zeros(n_cells + 1)
j_ie_edge = np.zeros(n_cells + 1)
for edge_idx in range(n_cells + 1):
    phi_ie_edge[edge_idx] = np.sum(quad_dict["w"] *
    psi_edge_matrix[:, edge_idx])
    j_ie_edge[edge_idx] = np.sum(quad_dict["w"] * quad_dict["mu"] *
    psi_edge_matrix[:, edge_idx])

results_dict = {
    "critical_width": a_next,
    "cell_average_angular_flux": psi_0moment_matrix,
    "cell_edge_angular_flux": psi_edge_matrix,
    "first_legendre_moment_angular_flux": psi_1moment_matrix,
    "cell_average_scalar_flux": 0.5 * (phi_R + phi_L),
    "cell_edge_scalar_flux": phi_ie_edge,
    "first_legendre_moment_scalar_flux": 0.5 * (phi_R - phi_L),
    "cell_average_current": 0.5 * (j_R + j_L),
    "cell_edge_current": j_ie_edge,
    "first_legendre_moment_current": 0.5 * (j_R - j_L),
    "phi_L": phi_L,
    "phi_R": phi_R,
    "j_L": j_L,
    "j_R": j_R,
```

```
        "inner_history": history,
        "outer_history": outer_history,
        "delta_r_0": delta_r_0,
        "delta_r_1": delta_r_1,
        "delta_R_0": delta_R_0,
        "delta_R_1": delta_R_1,
    }

    return results_dict


def eigenvalue_inner_loop(width_guess, quad_dict, problem, file_obj=None):
    """ inner loop solver for the k-eigenvalue solver """
    problem.region_boundaries = [0, width_guess]
    mesh_dict = mesh_builder.discretize_space(problem)

    dx = mesh_dict["dx"]
    sigma_t = mesh_dict["sigma_t"]
    sigma_s = mesh_dict["sigma_s"]
    sigma_f = mesh_dict["sigma_f"]
    nu_f = mesh_dict["nu_f"]
    q = mesh_dict["q"]

    n_cells = len(dx)
    phi_L = np.ones(n_cells)
    phi_R = np.ones(n_cells)

    epsilon = problem.convergence_criteria

    k_current = 1.0
    k_prev1 = 1.0
    k_prev2 = 1.0
```

```
inner_converged = False
iteration_idx = 0
iteration_history = []
error_phi_previous = 0

delta_r_0, delta_r_1, delta_R_0, delta_R_1 = None, None, None, None

psi_left_reflective = None

while not inner_converged:
    iteration_idx += 1

    phi_L_prev = phi_L.copy()
    phi_R_prev = phi_R.copy()
    phi_0moment_prev = 0.5 * (phi_L_prev + phi_R_prev)

    sigma_t_dx_over_3 = (1 / 3) * sigma_t * dx
    sigma_t_dx_over_6 = (1 / 6) * sigma_t * dx

    production_coeff = sigma_s + (sigma_f * nu_f / k_current)
    internal_source_diag = (1 / 6) * production_coeff * dx
    internal_source_off = (1 / 12) * production_coeff * dx

    external_source = (1 / 4) * q * dx

    b_source_L = (internal_source_diag * phi_L_prev +
    internal_source_off * phi_R_prev) + external_source
    b_source_R = (internal_source_off * phi_L_prev +
    internal_source_diag * phi_R_prev) + external_source

    (phi_L_new, phi_R_new, j_L, j_R, psi_left_reflective,
```

```
 psi_0moment_matrix, psi_1moment_matrix, psi_edge_matrix) =
 angular_mesh_sweep(
    mesh_dict, quad_dict,
    sigma_t_dx_over_3, sigma_t_dx_over_6,
    b_source_L, b_source_R,
    problem, psi_left_reflective
)

phi_0moment_new = 0.5 * (phi_L_new + phi_R_new)

numer = np.sum(sigma_f * nu_f * phi_0moment_new * dx)
denom = np.sum(sigma_f * nu_f * phi_0moment_prev * dx)
k_next = k_current * (numer / denom)

error_phi_L = np.max(np.abs(phi_L_new - phi_L_prev))
error_phi_R = np.max(np.abs(phi_R_new - phi_R_prev))
error_phi_current = max(error_phi_L, error_phi_R)

error_k_current = abs(k_next - k_current)

converged_phi = False
converged_k = False

if iteration_idx >= 2 and error_phi_previous > 0:
    rho_phi = error_phi_current / error_phi_previous
    if 0 < rho_phi < 1:
        threshold_phi = epsilon * ((1 / rho_phi) - 1)
        if error_phi_current < threshold_phi:
            converged_phi = True

if iteration_idx >= 3:
    denom_k = abs(k_current - k_prev2)
```

```
        if denom_k > 0:
            rho_k = error_k_current / denom_k
            if 0 < rho_k < 1:
                threshold_k = epsilon * ((1 / rho_k) - 1)
                if error_k_current < threshold_k:
                    converged_k = True

    if converged_phi and converged_k:
        inner_converged = True

    if inner_converged:
        phi_0moment_current = 0.5 * (phi_L + phi_R)
        phi_1moment_current = 0.5 * (phi_R - phi_L)

        phi_0moment_prev = 0.5 * (phi_L_prev + phi_R_prev)
        phi_1moment_prev = 0.5 * (phi_R_prev - phi_L_prev)

        q_0moment_integrated = q * dx
        q_1moment_integrated = np.zeros(n_cells)

        production = sigma_s + (sigma_f * nu_f / k_next)

        delta_r_0 = (j_R - j_L) + (sigma_t * phi_0moment_current * dx) -
        (production * phi_0moment_prev * dx) - q_0moment_integrated

        j_0moment = 0.5 * (j_L + j_R)
        delta_r_1 = 3.0 * (j_R + j_L - 2 * j_0moment) +
        (sigma_t * phi_1moment_current * dx) -
        (production * phi_1moment_prev * dx) - q_1moment_integrated

        delta_R_0 = (j_R - j_L) + ((sigma_t - production) *
        phi_0moment_current * dx) - q_0moment_integrated
```

```
        delta_R_1 = 3.0 * (j_R + j_L - 2 * j_0moment) +
        ((sigma_t - production) * phi_1moment_current * dx) - q_1moment_integrated

    iteration_history.append({
        "iteration": iteration_idx,
        "error_phi": error_phi_current,
        "error_k": error_k_current,
        "k_eff": k_next
    })

    error_phi_previous = error_phi_current
    k_prev2 = k_prev1
    k_prev1 = k_current
    k_current = k_next
    phi_L, phi_R = phi_L_new, phi_R_new

phi_0moment = 0.5 * (phi_L + phi_R)
phi_integral = np.sum(phi_0moment * dx)
phi_L /= phi_integral
phi_R /= phi_integral

# Return the extra matrices up to the outer solver loop
return k_current, phi_L, phi_R, j_L, j_R, iteration_history,
psi_0moment_matrix, psi_1moment_matrix, psi_edge_matrix,
delta_r_0, delta_r_1, delta_R_0, delta_R_1
```